Manual técnico del Carro FSI

Arquitectura, operación y evidencia de un vehículo de navegación asistida

Fundamentos de los Sistemas de Información

Profesor titular: Dr. Horacio Rostro González

Jaime Fernando Guerrero Segoviano
Melanie Contreras Solis
Vitto Eduardo Rangel Fortuna
Azul Abril Cadena Campos

Entrega: 1 de diciembre de 2026

Versión de fuentes: v1.0-23-gf810e3b · Commit: f810e3b

Contenido

1. Introducción **6**

1.1 Cómo interpretar la evidencia . . . 6

1.2 Antes de comenzar . . . 7

1.3 Recorrido recomendado de lectura . . . 7

1.4 Una orden de principio a fin . . . 7

1.5 Unidades y convenciones . . . 7

2. Arquitectura general **8**

2.1 Los tres nodos . . . 8

2.2 Alimentación . . . 8

2.3 Capas de seguridad . . . 8

2.4 Responsabilidad de cada enlace . . . 8

2.5 Qué sucede durante una ruta . . . 9

2.6 Señales, energía y referencia eléctrica . . . 9

2.7 Separación entre decisión y observación . . . 9

2.8 Secuencia de fallos para razonar . . . 9

3. Hardware y conexiones **11**

3.1 Componentes . . . 11

3.2 Conexiones de control . . . 11

3.3 Revisión de conexiones . . . 12

3.4 Inventario funcional . . . 12

3.5 Separación entre señales y potencia . . . 12

3.6 Identificación de los canales de motor . . . 13

3.7 Bus compartido de sensores . . . 13

3.8 Corriente, temperatura y autonomía . . . 13

4. Firmware del carro **14**

4.1 Control y perfiles . . . 14

4.2 Vigilancia y sensores . . . 15

4.3 Secuencia de trabajo . . . 15

4.4 Inicialización y orden de las tareas . . . 15

4.5 Cola y prioridad de detención . . . 16

4.6 Perfiles, balance y arranque . . . 16

4.7 Giro con giroscopio . . . 16

4.8 Lecturas de distancia y autoridad de parada . . . 17

5. Cámara Freenove **18**

5.1 Captura y transmisión . . . 18

5.2 Rendimiento y límites . . . 18
5.3 Servicios y gestión de cuadros . . . 18
5.4 Placa, sensor y formatos . . . 19
5.5 Dirección y conexión a la red . . . 19
5.6 Un cuadro frente a un flujo continuo . . . 19
5.7 Datos por celdas . . . 20
5.8 Leer los resultados del barrido . . . 20
5.9 Fallas y recuperación . . . 20
6. Protocolo de comunicación 20
6.1 Movimiento y parada . . . 21
6.2 Estado y configuración . . . 21
6.3 Interpretación de telemetría . . . 21
6.4 Direcciones y sesiones . . . 21
6.5 Orden mantenida y pulso de conexión . . . 22
6.6 Interpretar un ejemplo de sensores . . . 22
6.7 Consultas, confirmaciones y errores . . . 22
7. Núcleo de navegación en C 23
7.1 Ruta, calibración y pose . . . 23
7.2 Esquive y reincorporación . . . 23
7.3 Simulación . . . 23
7.4 Ajuste de movimiento . . . 24
7.5 Lectura de la simulación . . . 24
7.6 De una cuadrícula a movimientos . . . 24
7.7 Estados y transiciones . . . 24
7.8 Confirmación de obstáculos y pérdida de imagen . . . 25
7.9 Desvío y regreso a la ruta . . . 25
7.10 Datos que conserva la calibración . . . 25
8. Visión y percepción 27
8.1 Color, objetos y corredor . . . 27
8.2 Distancia frontal . . . 27
8.3 De la imagen a una decisión . . . 27
8.4 Formación de la cuadrícula de color . . . 28
8.5 Color, brillo y varios tonos . . . 28
8.6 Zonas y niveles de cercanía . . . 28
8.7 Modelo de objetos y vigencia . . . 29
8.8 Reglas básicas de fusión . . . 29
9. Aplicación Android 30

9.1 Pantallas principales . . . 30
9.2 Uso del video y percepción . . . 30
9.3 Preparación de una ruta . . . 31
9.4 Organización de la interfaz . . . 31
9.5 Conexión compartida . . . 31
9.6 Conexión y selección de perfil . . . 31
9.7 Trabajo entre Android y el núcleo . . . 32
9.8 Estados de las capturas . . . 32
10. Compilación, instalación y versión 33
10.1 Requisitos registrados . . . 34
10.2 Compilación y pruebas sin carga . . . 34
10.3 Identificar el producto antes de actuar . . . 34
10.4 Diferencia entre compilar y cargar . . . 34
10.5 Interpretación de errores de construcción . . . 35
10.6 Manejo de configuración sensible . . . 35
11. Operación y seguridad física 35
11.1 Preparación y conexión . . . 35
11.2 Durante la operación . . . 36
11.3 Preparar el entorno . . . 36
11.4 Encender y verificar sin movimiento . . . 36
11.5 Conducir en manual . . . 37
11.6 Preparar y ejecutar una ruta . . . 37
11.7 Cierre de una sesión . . . 37
12. Pruebas y resultados 37
12.1 Pruebas físicas registradas . . . 38
12.2 Software y simulación . . . 38
12.3 Pendientes físicos . . . 38
12.4 Qué demuestra cada clase de ensayo . . . 38
12.5 Lectura de las cifras de simulación . . . 39
12.6 Condiciones de las pruebas físicas anteriores . . . 39
12.7 Cómo registrar una repetición futura . . . 39
12.8 Preparar una ficha de ensayo . . . 40
12.9 Ensayo propuesto de movimiento recto . . . 40
12.10 Ensayo propuesto de giro . . . 40
12.11 Ensayo propuesto de video y control . . . 41
12.12 Ensayo propuesto de parada frontal . . . 41
12.13 Análisis de una serie de resultados . . . 41

12.14 Comparación entre revisiones . . . 41

13. Límites, riesgos y trabajo futuro 42

13.1 Alcance del entorno de operación . . . 42

13.2 Precisión y repetibilidad . . . 42

13.3 Evidencia pendiente y prioridad . . . 43

13.4 Cambios futuros y regresiones . . . 43

14. Proceso de desarrollo 43

14.1 Separar el control del hardware . . . 44

14.2 Incorporar sensores y calibrar . . . 44

14.3 Cámara y procesamiento local . . . 44

14.4 Diagnóstico de reincorporación . . . 45

14.5 Alternativas y compromisos . . . 45

15. Solución de problemas 46

15.1 Red y video . . . 46

15.2 Movimiento y sensores . . . 46

15.3 Método de diagnóstico . . . 46

15.4 No aparece la red del carro . . . 46

15.5 La red existe, pero no hay control . . . 47

15.6 Hay control, pero falta imagen . . . 47

15.7 El carro no se mueve . . . 47

15.8 El carro se desvía . . . 47

15.9 La orientación cambia con el carro quieto . . . 48

15.10 El sensor frontal no responde o parece inconsistente . . . 48

15.11 La ruta no termina donde se esperaba . . . 48

16. Anexos 48

16.1 Glosario . . . 48

16.2 Comandos principales . . . 48

16.3 Pines de la DevKit . . . 49

16.4 Fuentes por tema . . . 49

16.5 Pendientes . . . 49

16.6 Glosario ampliado: electrónica y energía . . . 49

16.7 Glosario ampliado: comunicaciones . . . 50

16.8 Glosario ampliado: movimiento y percepción . . . 51

16.9 Glosario ampliado: pruebas . . . 51

17. Referencia técnica de archivos 52

17.1 Firmware: arranque, configuración y seguridad . . . 52

17.2 Firmware: sensores y giro . . . 52
17.3 Cámara y recuperación . . . 53
17.4 Núcleo C: ruta, calibración y navegación . . . 54
17.5 Núcleo C: percepción y video . . . 55
17.6 Compilación y conexión entre C y Android . . . 57
17.7 Android: archivos Kotlin de aplicación . . . 57
17.8 Android: declaraciones para el núcleo nativo . . . 61
17.9 Pruebas, recursos y archivos auxiliares . . . 62
17.10 Herramientas de apoyo, archivo por archivo . . . 63
17.11 Pruebas del núcleo, archivo por archivo . . . 64
17.12 Lectura guiada: del comando a los motores . . . 65
17.13 Lectura guiada: sensores y giro . . . 66
17.14 Lectura guiada: de la ruta a la decisión . . . 66
17.15 Lectura guiada: imagen, color y fusión . . . 66
17.16 Lectura guiada: Kotlin, JNI y C . . . 67

1. Introducción

El Carro FSI es un vehículo de cuatro ruedas controlado por una aplicación Android y una placa ESP32. Permite conducirlo de forma manual o preparar una ruta en una cuadrícula para que el sistema intente seguirla con supervisión de una persona. Una cámara independiente transmite video y los sensores del carro aportan información de orientación y distancia frontal.

Este manual describe la arquitectura, las conexiones, el funcionamiento del software, la operación y la evidencia disponible. Está dirigido a quien necesita comprender, mantener o presentar el proyecto sin haber participado en su desarrollo. La descripción del comportamiento previsto no sustituye una prueba del vehículo físico.

Para comenzar, conviene leer la arquitectura y el hardware. Los capítulos de firmware, cámara, comunicación, navegación y percepción explican las responsabilidades de cada parte. La operación y las pruebas reúnen las condiciones de uso y los resultados; los anexos concentran comandos, pines y referencias al repositorio.

1.1 Cómo interpretar la evidencia

VERIFICADO identifica un resultado observado en el carro u otro dispositivo físico, bajo las condiciones indicadas. **SIMULADO** identifica un resultado obtenido con un modelo de software. **NO MEDIDO** señala una característica para la que no existe una medición física suficiente. **PENDIENTE** señala una comprobación o dato necesario que todavía falta. Una simulación ayuda a revisar la lógica, pero no demuestra por sí sola el comportamiento del carro en el piso.

1.2 Antes de comenzar

Conviene separar tres preguntas: qué está diseñado para hacer el sistema, qué muestra una prueba de software y qué ocurrió realmente en el vehículo. El código responde sobre la implementación de una revisión concreta. Los registros de prueba describen ejecuciones concretas. Una observación actual requiere revisar el equipo y anotar sus condiciones.

El término «autónomo» se emplea aquí para el seguimiento de una ruta por software. No significa que el carro pueda circular sin vigilancia, reconocer cualquier obstáculo o garantizar que alcanzará una posición exacta. El operador debe preparar el área y mantener la capacidad de detener la marcha.

1.3 Recorrido recomendado de lectura

Para una primera utilización, lea la arquitectura, las precauciones de hardware y el procedimiento de operación. Si aparece un problema, consulte el diagnóstico y registre lo que observa antes de cambiar varias cosas a la vez. Para modificar el software, continúe con los capítulos de firmware, protocolo, navegación, visión y aplicación.

Para evaluar los resultados, lea pruebas y límites conjuntamente. Una cifra de simulación se interpreta con el modelo y las condiciones del experimento; no se transforma en una medición física por el hecho de aparecer en una gráfica o tabla. Los anexos ayudan a localizar nombres y conceptos cuando se revisa el repositorio.

1.4 Una orden de principio a fin

Cuando el operador pide avanzar, la aplicación prepara una orden y la envía por el canal de control. El controlador la interpreta y modifica las señales que llegan al driver. El driver conmuta la alimentación de los motores, que producen el movimiento de las ruedas. En sentido inverso, el controlador comunica información de sensores y la cámara transmite imágenes por un canal independiente.

Cada paso puede fallar por una causa diferente. Que una pantalla responda al toque no demuestra que la orden haya llegado; que una orden llegue tampoco demuestra que las ruedas se muevan. En la práctica se observan por separado la conexión, la respuesta del controlador y el resultado físico.

1.5 Unidades y convenciones

Las distancias se expresan en centímetros cuando se habla de ruta y en las unidades indicadas por cada interfaz de sensor cuando se consulta una lectura. El tiempo aparece en milisegundos para intervalos de control y en minutos para sesiones. Un grado expresa un ángulo, no una distancia. Antes de comparar cifras, hay que comprobar que representen la misma magnitud y empleen las mismas unidades.

La palabra «estimado» significa calculado a partir de información disponible y un modelo; no implica que se haya observado la posición real con un instrumento. «Configurado» indica un valor presente en software. «Medido» requiere un procedimiento y un resultado registrados. Estas distinciones permiten explicar el proyecto sin atribuirle precisión que no se ha demostrado.

Dónde está en el código: `firmware_carro.ino`, `navegador.c`, `MainActivity.kt`.

2. Arquitectura general

El sistema distribuye el trabajo entre tres nodos: el controlador del carro, la cámara y el teléfono. Esta separación permite que la adquisición y compresión de video se realicen en una placa distinta de la que controla motores y sensores. La aplicación reúne la interacción con el operador y ejecuta el núcleo de navegación.

2.1 Los tres nodos

La ESP32 DevKit del carro crea la red local y recibe las órdenes de movimiento. Controla el puente H, un circuito que permite invertir el sentido de los motores, y consulta el giroscopio y el sensor de distancia. Su dirección de red es 192.168.4.1.

La placa Freenove con cámara se conecta a esa red como cliente, con la dirección 192.168.4.2. Captura imágenes y puede transmitir video. El teléfono Android también se conecta a la red del carro: presenta los controles, recibe información y ejecuta la lógica de navegación en C mediante la aplicación.

La red se llama CarroFSI. La comunicación de control usa WebSocket, un canal bidireccional persistente; el video sigue un canal separado. Por eso una falla de video y una pérdida del enlace de control deben diagnosticarse por separado.

2.2 Alimentación

La DevKit se alimenta por USB desde una batería portátil. La cámara Freenove usa otra batería portátil propia. Los motores reciben energía de un portapilas de cuatro pilas AA a través del driver. El negativo de las pilas, la tierra del driver y la tierra de la DevKit comparten referencia para las señales de control. La cámara se comunica por WiFi con el resto del sistema.

2.3 Capas de seguridad

La persona conserva la responsabilidad de supervisar el espacio y poder detener el vehículo. La aplicación puede enviar una orden de parada; el firmware también vigila un temporizador de comunicación. Un sensor láser frontal puede detener el avance en el perfil AUTO cuando se cumplen las condiciones configuradas. Esa protección tiene un campo de observación limitado y puede desactivarse, por lo que no reemplaza la supervisión.

El movimiento recto se estima a partir del tiempo, no de una medición directa de distancia mediante encoders. El giroscopio ayuda a estimar el rumbo; no mide directamente el desplazamiento lateral. La corrección de pose y reincorporación de J15 cuenta con resultados de simulación, pero su validación física permanece **NO MEDIDA**.

2.4 Responsabilidad de cada enlace

El enlace de control transporta órdenes y respuestas entre la aplicación y la DevKit. El flujo de video transporta imágenes entre el teléfono y la cámara. Ambos utilizan la misma red local, pero son servicios distintos: el estado de uno no prueba el del otro. Esta separación permite diagnosticar, por ejemplo, un carro controlable sin imagen o una imagen disponible mientras falla el control.

La comunicación de los sensores montados en el carro se realiza localmente con la DevKit. I2C comparte las líneas de datos y reloj entre dispositivos compatibles, mientras que el puente H recibe señales de mando para cada lado de los motores. Un fallo del bus puede afectar a las lecturas aunque la red inalámbrica siga funcionando.

2.5 Qué sucede durante una ruta

El operador construye una secuencia de posiciones en la cuadrícula. El núcleo interpreta esa secuencia, determina la maniobra correspondiente y actualiza una pose estimada: posición y orientación calculadas. La aplicación traduce las decisiones del núcleo en órdenes para el carro y aporta información de los sensores y de la cámara cuando están disponibles.

La DevKit sigue siendo responsable de ejecutar las órdenes y de sus protecciones locales. La cámara, por su parte, no maneja el puente H. Esta división aclara dónde investigar un problema: una orden equivocada puede originarse en la planificación o en su traducción; una orden correcta sin movimiento puede apuntar al enlace, el firmware, la alimentación o la mecánica.

2.6 Señales, energía y referencia eléctrica

Una señal de control indica al driver lo que debe hacer; no es la fuente de energía principal de los motores. Las pilas proporcionan esa energía a través del driver. La tierra común entre el controlador y el driver permite que el nivel de las señales tenga una referencia compartida. La conexión de tierras debe comprobarse con el sistema apagado y de acuerdo con el esquema del montaje.

La cámara tiene una batería portátil distinta y se comunica por WiFi. Que ambas placas usen USB no convierte sus baterías en una sola fuente. Tampoco debe suponerse que las baterías y el portapilas tienen el mismo estado de carga; un circuito puede funcionar mientras otro queda sin energía.

2.7 Separación entre decisión y observación

Una imagen es una observación del entorno; una región marcada por el procesamiento es una interpretación de esa imagen. La distancia del láser es otra observación, limitada a la zona que cubre su orientación. La fusión combina información disponible según reglas del software, pero no garantiza una visión completa de todo el espacio alrededor del carro.

La pose de navegación también es una interpretación. En un tramo recto, el programa estima la distancia con el tiempo y una calibración; el giroscopio aporta información del giro. Si una rueda patina, la distancia estimada puede diferir de la real sin que el sistema conozca inmediatamente esa diferencia.

2.8 Secuencia de fallos para razonar

Considere una ruta iniciada con video ausente. Primero hay que distinguir si el modo elegido exige cámara y qué protección se activa con esa configuración. Después se verifica la alimentación de la cámara, su conexión a la red y la recepción de imágenes. No es correcto concluir que el puente H está averiado solo porque no aparece video.

Si hay video pero la orden de avance no produce movimiento, la investigación cambia: conexión de control, estado del carro, alimentación de motores, señales al driver y mecánica son áreas distintas de comprobación. En ambos casos se detiene el carro antes de manipular cables.

La Figura 1 presenta: Arquitectura de tres nodos y alimentación independiente de las placas.

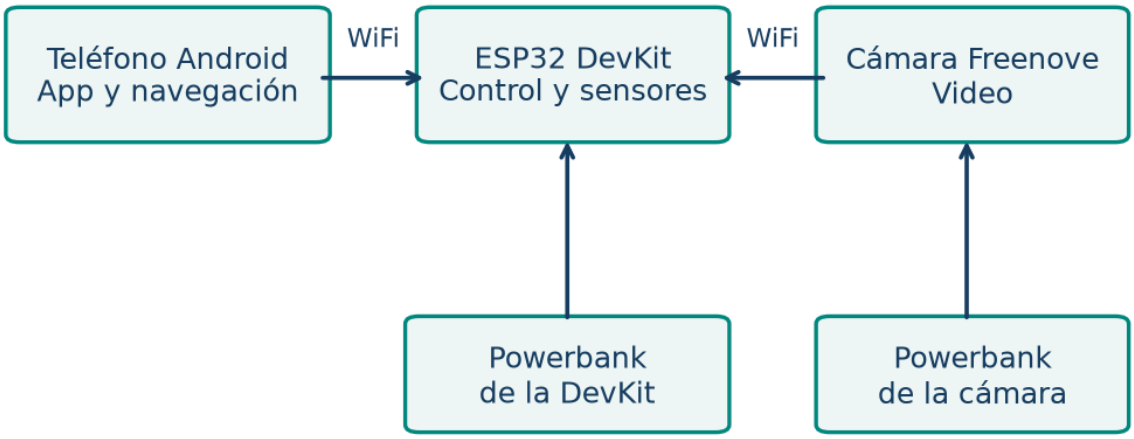


Figura 1. Arquitectura de tres nodos y alimentación independiente de las placas.

La Figura 2 presenta: Alimentación y referencia común entre DevKit, driver y pilas.

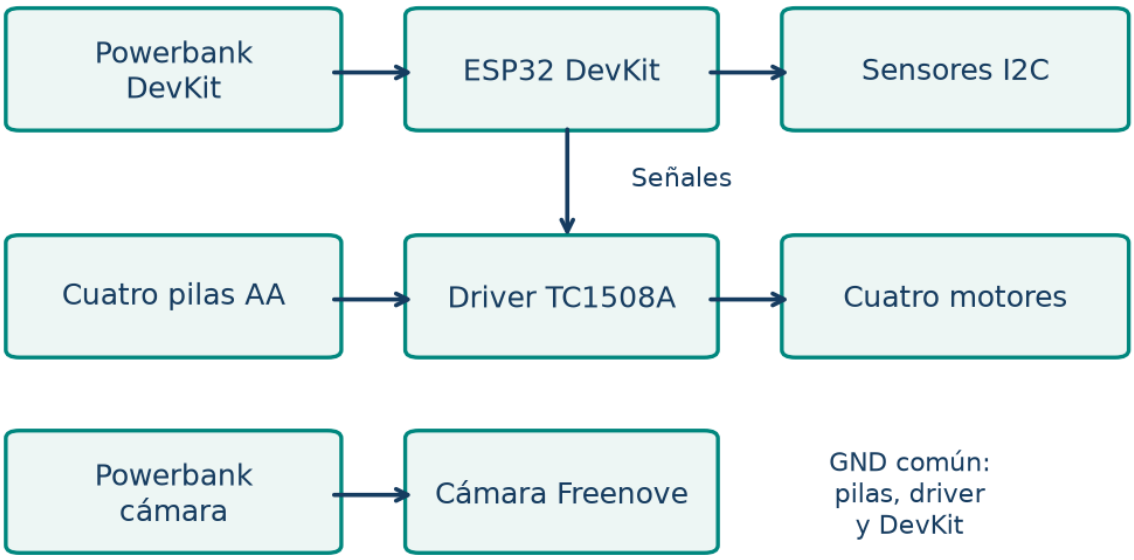


Figura 2. Alimentación y referencia común entre DevKit, driver y pilas.

La Figura 3 presenta: Capas de supervisión y condiciones de parada.

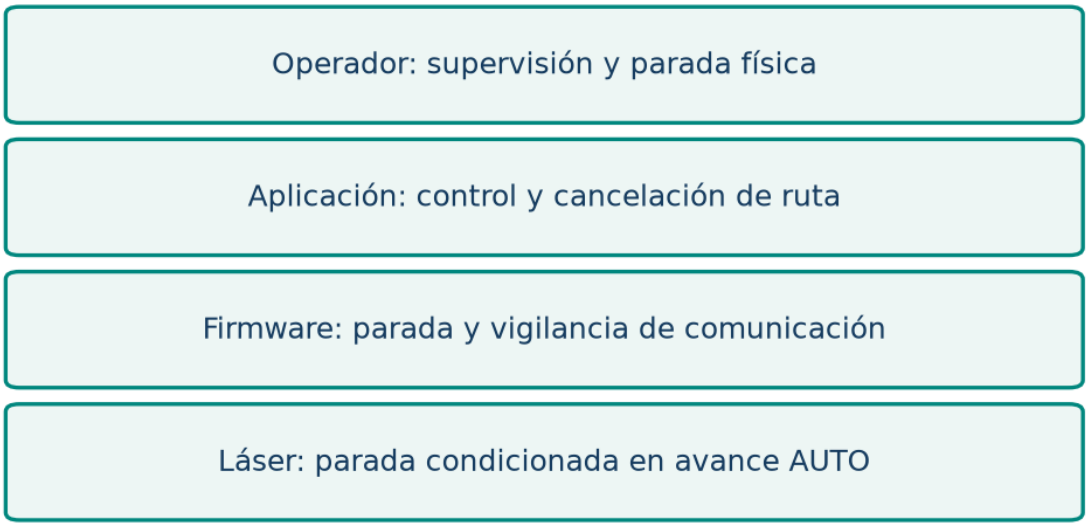


Figura 3. Capas de supervisión y condiciones de parada.

Dónde está en el código: `firmware_carro.ino`, `config.h`, `camara_prueba.ino`, `navegador.c`.

3. Hardware y conexiones

El carro reúne tracción, sensores y dos placas con funciones distintas. Este capítulo identifica las conexiones principales y las precauciones necesarias para revisarlas; los detalles de protocolo se explican más adelante.

3.1 Componentes

El chasis tiene cuatro motores TT, dos por cada canal del controlador TC1508A. La ESP32 DevKit controla el driver y consulta dos sensores: el MPU6050, utilizado como giroscopio, y el VL53L0X, sensor láser de distancia frontal. La cámara es una Freenove ESP32-WROVER con sensor GC0308. El teléfono Android ejecuta la aplicación.

La DevKit y la cámara disponen de powerbanks independientes. Cuatro pilas AA alimentan el circuito de motores. Antes de intervenir el cableado deben retirarse las pilas y desconectarse las baterías USB.

3.2 Conexiones de control

La DevKit utiliza GPIO 27 y 26 para las entradas del canal izquierdo del driver, y GPIO 25 y 33 para el canal derecho. Los sensores comparten el bus I2C, una conexión de datos entre circuitos, mediante SDA en GPIO 21 y SCL en GPIO 22. El firmware configura GPIO 2 para un pulso periódico de mantenimiento de la batería portátil; su conexión y eficacia físicas están **NO MEDIDAS**.

El positivo del portapilas se conecta a la alimentación del driver, no a un pin de la DevKit. El negativo del portapilas, la tierra del driver y la tierra de la DevKit deben compartir referencia para las señales de control. El consumo de los motores, incluido el de bloqueo,

está **NO MEDIDO**; las capacidades nominales del driver no demuestran que las corrientes reales sean seguras.

3.3 Revisión de conexiones

El MPU6050 y el VL53L0X comparten SDA y SCL; cada dispositivo responde a su propia dirección en el bus. Las conexiones deben estar firmes y el cableado no debe rozar las ruedas. Para revisar el montaje se desconectan las fuentes de alimentación antes de manipular los conductores; la comprobación de continuidad permite buscar cortocircuitos antes de energizar el sistema.

El driver tiene dos canales y cada canal alimenta un par de motores en paralelo. Si el sentido de un lado no corresponde a la orden, se revisa la polaridad del par y la correspondencia de entradas. No se debe inferir que el carro está seguro para operar con ruedas bloqueadas a partir de una prueba con ruedas libres.

3.4 Inventario funcional

La DevKit proporciona el punto de acceso inalámbrico, las salidas de control y el bus de sensores. El módulo TC1508A recibe señales de la DevKit y entrega potencia a los motores. Los cuatro motores TT se agrupan por lado: dos motores izquierdos conectados a un canal y dos derechos al otro. La placa de cámara Freenove es un segundo microcontrolador con su propio sensor y memoria; el teléfono aporta la interfaz y el cálculo de navegación.

Los elementos de alimentación son dos baterías portátiles USB, una para cada placa, y un portapilas de cuatro celdas AA para el driver. Los cables USB de datos se usan para las tareas de carga de programa y diagnóstico; una conexión adecuada para alimentación no garantiza que el cable transporte datos. Para una lista de compra con marcas, capacidades y precios no hay inventario completo respaldado: esos datos permanecen **PENDIENTES**.

3.5 Separación entre señales y potencia

Un GPIO es una entrada o salida digital del microcontrolador. No es una fuente adecuada para alimentar directamente los motores. El puente H recibe señales de mando de bajo consumo y conmuta la alimentación que procede del portapilas. La dirección de giro se decide por cuál entrada del canal recibe la señal; el valor PWM determina la fracción de tiempo activo.

El positivo de las cuatro pilas se lleva a VDD del TC1508A. El negativo de las pilas debe compartir referencia con la tierra del driver y la de la DevKit. La necesidad de esa referencia se limita a los circuitos que intercambian señales eléctricas: la cámara se comunica por WiFi y dispone de su propia alimentación. Este esquema no establece por sí mismo un detalle de cableado interno de la cámara.

La separación de alimentación busca evitar que los cambios de corriente de tracción alteren la alimentación del controlador. No constituye una medición de ruido eléctrico ni garantiza ausencia de reinicios. Para investigar un reinicio deben registrarse la condición de movimiento, la carga de las baterías y el resultado observado, en lugar de atribuirlo automáticamente a una causa.

3.6 Identificación de los canales de motor

El canal izquierdo usa GPIO 27 para la entrada A y GPIO 26 para la B; en la configuración del proyecto el avance izquierdo utiliza la entrada B. El derecho usa GPIO 25 para la entrada A y GPIO 33 para la B; el avance derecho utiliza la A. Esos nombres describen el mapeo lógico del firmware. Los rótulos impresos en el módulo concreto deben comprobarse contra sus conexiones antes de modificar cables.

Una inversión de polaridad en un par puede producir avance en un lado y retroceso en el otro. Una rueda que no gira también puede deberse a un conector suelto o un problema mecánico; no debe corregirse cambiando valores de software sin revisar primero el montaje. La inspección se realiza con las fuentes desconectadas. Una vez revisado el cableado, la guía de conexión propone pruebas iniciales con ruedas elevadas y control de una sola etapa a la vez.

3.7 Bus compartido de sensores

SDA es la línea de datos del bus I2C y SCL es la línea de reloj. Ambos módulos se conectan a las mismas dos señales, junto con 3.3 V y tierra. El código configura una frecuencia de 400 kHz; el MPU6050 responde en la dirección 0x68 y el VL53L0X en 0x29. Una dirección identifica al dispositivo en ese bus, no es una dirección de red WiFi.

La comunicación puede fallar aunque un LED de alimentación esté encendido. El LED demuestra que el módulo recibe alguna alimentación, pero no demuestra continuidad de SDA y SCL ni lectura válida. Un registro del proyecto describe un cable SCL flojo que dejó de responder al giroscopio. Ante un fallo se revisan alimentación, referencia común, continuidad y fijación de los conductores antes de interpretar el sensor como defectuoso.

El proyecto documenta una orientación prevista del giroscopio con el eje Z hacia arriba y la referencia frontal del módulo hacia adelante. La orientación y el signo importan porque el firmware interpreta el ángulo para decidir el sentido de giro. El láser se monta hacia el frente; su altura y ángulo condicionan qué objetos intercepta el haz. Un cambio físico de montaje requiere repetir las comprobaciones de orientación y detección.

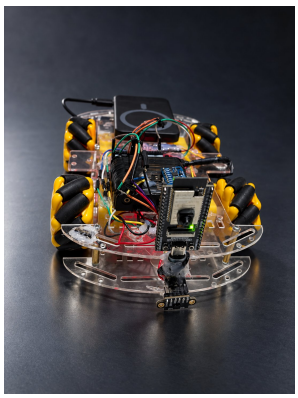
3.8 Corriente, temperatura y autonomía

Los dos motores de un lado comparten un canal del driver, por lo que el esfuerzo del canal depende de la corriente conjunta. El registro de conexiones cita límites nominales del componente; no contiene una caracterización suficiente del consumo real del conjunto bajo todas las cargas. El arranque, la fricción y el bloqueo pueden producir condiciones diferentes de una rueda girando sin carga.

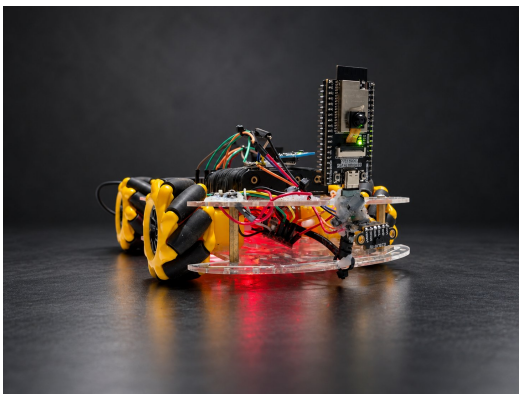
Antes de decidir que un componente tiene margen suficiente, deben medirse las corrientes pertinentes con un instrumento y procedimiento adecuados, registrando tensión, configuración, duración y carga. La temperatura del driver y la autonomía tampoco cuentan con una medición suficiente en el repositorio. Estas magnitudes no deben sustituirse por valores típicos de otro carro.

El firmware configura GPIO 2 para emitir un pulso de 50 ms cada dos segundos. El propósito es mantener activa una batería portátil de baja carga, pero no se conoce con certeza qué carga física está conectada al pin ni si esa batería necesita el pulso. La señal programada es un dato de configuración; su efecto real permanece **NO MEDIDO**.

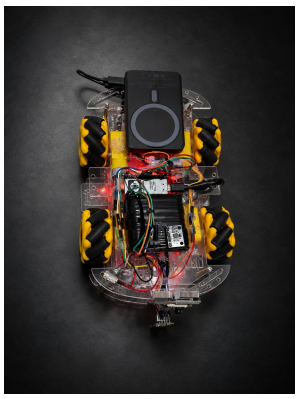
La Figura 4 muestra las vistas correspondientes.



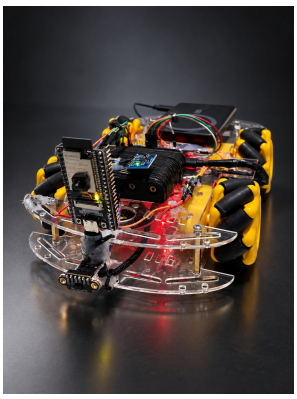
Vista frontal



Vista lateral



Vista superior



Vista en perspectiva

Figura 4. Cuatro vistas del carro.

Dónde está en el código: `config.h`, `firmware_carro.ino`, `sensores.c`.

4. Firmware del carro

El firmware recibe las órdenes de la aplicación y las convierte en señales para los motores. También consulta sensores y aplica límites de seguridad. Separar la lógica de control de las funciones que acceden al hardware permite probar parte del comportamiento en una computadora.

4.1 Control y perfiles

Las órdenes llegan por WebSocket y se incorporan a una cola; el ciclo principal procesa esa cola y actualiza los motores. La modulación por ancho de pulso, PWM, regula la potencia mediante una señal que cambia rápidamente entre encendido y apagado. La configuración usa 1500 Hz y resolución de ocho bits.

Hay perfiles MANUAL y AUTO. Los valores predeterminados son 252 a la izquierda y 255 a la derecha en MANUAL, y 207 y 210 en AUTO. Esos valores pueden cambiar por ajustes guardados. El comentario de configuración registra un desvío de 3 cm en una prueba de 150 cm con el ajuste MANUAL; el balance AUTO derivado permanece **NO MEDIDO** en el carro.

4.2 Vigilancia y sensores

La vigilancia de comunicación está configurada para detener los motores cuando transcurren más de 1000 ms sin renovar el temporizador; un mensaje PING no lo renueva. El firmware también renueva ese temporizador para algunos mensajes que no son movimiento y para texto desconocido, por lo que esta protección no equivale a validar cada orden recibida. Un vigilante de hardware de 5000 ms busca recuperar el sistema si se bloquea el ciclo principal. Durante giros por ángulo se aplican reglas propias de duración y cancelación.

El MPU6050 aporta velocidad angular para estimar el giro. El VL53L0X mide distancia frontal; en AUTO y durante el avance, dos lecturas válidas inferiores a 120 mm activan la parada configurada. La autoridad del láser puede desactivarse y no cubre obstáculos fuera de su campo de observación.

La duración del ciclo no debe suponerse siempre inferior a 5 ms: las pruebas registraron excesos con la aplicación conectada. Las muestras son breves y no establecen por sí solas la causa.

4.3 Secuencia de trabajo

Al iniciar, el firmware configura las salidas de motores, la red y los sensores, y parte de un estado detenido. El ciclo principal revisa los sensores y procesa las órdenes recibidas. La cola evita que el procesamiento de red y el control de motores modifiquen simultáneamente el estado de movimiento desde contextos distintos.

El perfil AUTO limita la velocidad para las rutas, mientras MANUAL conserva el máximo configurado para conducción directa. Al cambiar entre perfiles, la respuesta de confirmación permite a la aplicación comprobar que el carro aceptó el modo solicitado. Los ajustes de balance pueden guardarse y prevalecer sobre los valores iniciales.

El giroscopio estima ángulo a partir de la velocidad angular; necesita una referencia de sesgo con el carro quieto. El láser no valida por sí mismo el espacio detrás del carro. Una orden de retroceso debe considerarse con el área posterior despejada.

4.4 Inicialización y orden de las tareas

Al encender, la placa abre la comunicación serie y registra la versión compilada. Configura el procesador, los canales PWM y el estado inicial de los motores; después prepara el bus I2C y los sensores, la vigilancia de hardware y el punto de acceso. El perfil inicial es AUTO. El servidor ofrece una página de control y un extremo WebSocket que la aplicación utiliza para intercambiar mensajes.

En el ciclo principal se atienden, entre otras tareas, la lectura de sensores, el avance de un giro por ángulo, la cola de comandos, la vigilancia de comunicación y el envío de estado. El orden tiene una finalidad: el acceso a sensores y las decisiones que cambian motores se concentran en el ciclo de control, mientras la recepción de red coloca mensajes para procesarlos. Un STOP recibido debe llegar a ese procesamiento sin quedar descartado por una ráfaga ordinaria.

La duración de una vuelta del ciclo puede consultarse mediante el diagnóstico LOOP. Un máximo aislado no describe toda la distribución de tiempos. En los ensayos históricos se observaron excesos de 5 ms con la aplicación conectada; las muestras no aislaron todas

las causas. Por ello la constante de 5 ms es un objetivo de diagnóstico, no una garantía comprobada en todas las condiciones.

4.5 Cola y prioridad de detención

La recepción WebSocket y el ciclo principal no realizan exactamente el mismo trabajo. La recepción incorpora el texto a una cola de capacidad limitada y el ciclo lo ejecuta. Si la cola se llena, la política da preferencia a conservar STOP y EBRAKE; primero puede descartar PING antiguos o mensajes que no sean de parada. Una prueba de estrés histórica de 100000 comandos registró cero STOP perdidos en ese ensayo.

El resultado anterior comprueba una propiedad de la prueba de software, no la latencia de frenado de extremo a extremo del carro real. El trayecto físico incluye recepción inalámbrica, ejecución, desactivación de señales y movimiento residual del chasis. Para comparar distancias de parada deben registrarse velocidad, superficie, alimentación y distancia medida en cada repetición.

4.6 Perfiles, balance y arranque

Cada perfil dispone de un valor máximo y un factor de balance. El factor reduce el lado izquierdo con relación al derecho; no incrementa la potencia por encima del máximo. Para MANUAL, el ajuste predeterminado de 988 milésimas sobre 255 produce aproximadamente 252 en el lado izquierdo. Para AUTO, el ajuste de 986 sobre 210 produce 207. Esos valores son puntos de partida; la memoria no volátil puede contener ajustes guardados distintos.

Durante el arranque de un avance o retroceso, la configuración aplica potencia plena por 250 ms antes de pasar al perfil. La razón documentada es que los motores de ese montaje podían zumbar sin iniciar movimiento con una potencia menor desde reposo. El intervalo no demuestra que todas las baterías y superficies producirán la misma aceleración; la calibración debe representar el montaje usado.

El pivote utiliza un lado de tracción activo y el otro detenido. Debido a la geometría y a las diferencias entre lados, un giro izquierdo no tiene necesariamente la misma duración ni el mismo desplazamiento del centro que un giro derecho. La orientación durante los giros por ángulo se observa con el giroscopio y se informa mediante la respuesta correspondiente.

4.7 Giro con giroscopio

El MPU6050 entrega velocidad angular. El firmware estima el ángulo integrando lecturas sucesivas después de restar el sesgo; por eso es necesario un arranque o calibración con el carro quieto. La configuración solicita 250 muestras para el cálculo de sesgo y un periodo nominal de 10 ms por muestra. El periodo configurado no prueba que cada lectura física llegue exactamente en ese intervalo.

Una orden GIRAR solicita un ángulo con signo: positivo para izquierda y negativo para derecha según la convención del proyecto. La lógica controla la maniobra, detiene el motor y espera un periodo de asentamiento para observar el resultado final. La anticipación aprendida intenta compensar el movimiento que continúa después de retirar la señal. La respuesta puede indicar éxito, tiempo agotado o error; un resultado

de éxito corresponde a la lógica y tolerancias implementadas, no a una garantía de posición absoluta.

Una muestra histórica quedó contaminada cuando se sujetó el carro durante un giro: la anticipación aprendida cambió con ese resultado. Para una nueva calibración conviene registrar cada giro y evitar intervenir físicamente durante la maniobra, salvo que sea necesario detenerla. Si un resultado es inválido, debe identificarse antes de utilizarlo como evidencia de precisión.

4.8 Lecturas de distancia y autoridad de parada

El VL53L0X mide hacia adelante mediante un principio de tiempo de vuelo de luz. El firmware consulta el sensor periódicamente y distingue mediciones válidas, ausencia de eco y fallas de comunicación. La configuración de parada en AUTO requiere dos lecturas válidas consecutivas menores de 120 mm mientras el carro avanza. Cuando esa condición se cumple, se detienen los motores y se informa el evento.

La opción de autoridad del láser puede desactivarse; una lectura disponible no implica necesariamente que esté habilitada la parada. El sensor no observa el espacio posterior ni todo el ancho del vehículo. Además, la distancia recorrida después de la orden de parada depende de la inercia, el piso y otros tiempos. El umbral configurado no es una distancia de frenado medida.

La Figura 5 presenta: Recorrido general de una orden de movimiento.

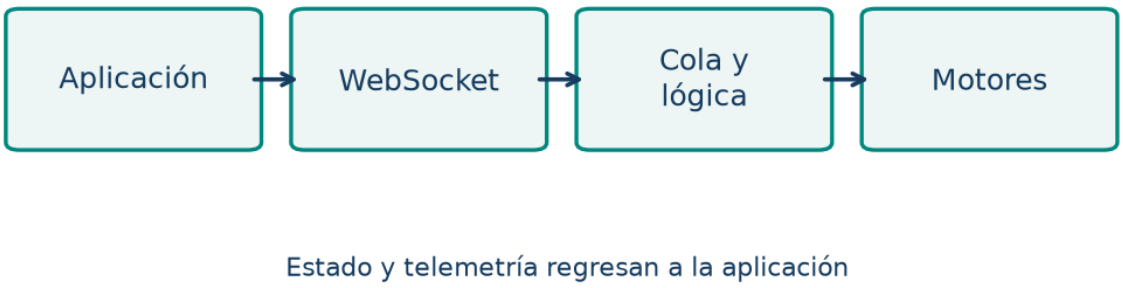


Figura 5. Recorrido general de una orden de movimiento.

Dónde está en el código: `config.h`, `carro.c`, `firmware_carro.ino`, `sensores.c`, `imu.c`, `tof.c`, `giro.c`.

5. Cámara Freenove

La cámara es un nodo separado que se conecta a la red creada por el carro. Su función es proporcionar imágenes o datos resumidos al teléfono, sin encomendar la compresión de video a la DevKit que controla la tracción.

5.1 Captura y transmisión

El sensor GC0308 entrega imágenes sin compresión JPEG de hardware. La placa comprime los cuadros por software y ofrece un flujo de imágenes JPEG mediante HTTP. También existen modos para obtener un cuadro RGB565 sin comprimir y para transmitir por UDP un resumen de colores por celdas. UDP es un protocolo de envío de datagramas que no confirma por sí mismo la entrega.

La configuración predeterminada del código usa HQVGA de 240 × 176 píxeles, RGB565, reloj de 20 MHz y calidad JPEG 50. La cámara emplea la dirección 192.168.4.2 y necesita conectarse a la red del carro para que el teléfono acceda a sus servicios.

5.2 Rendimiento y límites

En un barrido local sin cliente se midieron 100 cuadros por combinación. Con 20 MHz y RGB565, HQVGA registró 12.4 cuadros por segundo y 137 ms de edad del cuadro hasta terminar la compresión; QVGA registró 8.8 cuadros por segundo y 252 ms con calidad 50. Son resultados de ese ensayo, no una medición del retraso total hasta la pantalla del teléfono.

El firmware intenta recuperar la conexión y reiniciarse en determinadas fallas. La transmisión y la percepción siguen dependiendo de la red, la iluminación, la orientación de la cámara y el procesamiento posterior. El comportamiento de la cámara montada durante el movimiento requiere validación física.

5.3 Servicios y gestión de cuadros

El servicio `/stream` entrega una sucesión de imágenes JPEG; `/capture` entrega una captura y `/raw` expone un cuadro RGB565. La ruta `/stats` informa datos de funcionamiento para diagnóstico. El modo de celdas resume los colores de bloques de la imagen y los transmite por el puerto UDP 4210. La aplicación utiliza la dirección de la cámara configurada en Ajustes.

La captura y la transmisión se organizan con una casilla para el cuadro más reciente. Si el consumidor no alcanza el ritmo de captura, se descartan cuadros anteriores en lugar de acumular una cola cada vez más atrasada. Esto reduce acumulación de imágenes viejas, pero no elimina la latencia de extremo a extremo.

La placa intenta reconectarse cuando pierde la red y contempla reinicios ante determinadas fallas del servicio. Una ausencia de video puede tener varias causas: alimentación, conexión WiFi, una dirección incorrecta o un cliente de video anterior. Antes de interpretar un resultado de percepción, debe comprobarse que los cuadros estén llegando y que correspondan a la fuente seleccionada.

5.4 Placa, sensor y formatos

La Freenove ESP32-WROVER v3.0 incorpora un sensor GC0308 y memoria adicional PSRAM. Es una placa diferente de la DevKit de tracción, aunque ambas pertenezcan a la familia ESP32. Cada una tiene su propio programa, sus propias conexiones y su batería portátil. El teléfono no controla los motores mediante los pines de la cámara: envía comandos por la red a la DevKit.

El sensor del montaje entrega imágenes en formatos como RGB565; la compresión JPEG la realiza la placa por software. JPEG reduce el tamaño del cuadro a costa de pérdida de información. RGB565 representa colores con un número fijo de bits por píxel y se utiliza como entrada de procesamiento. El modo adecuado depende de si se necesita visualización, acceso al cuadro sin compresión JPEG o un resumen de color por celdas.

La resolución determina cuántos píxeles se procesan. HQVGA en esta configuración corresponde a 240 por 176 píxeles y QVGA a 320 por 240. Un cuadro más grande contiene más muestras de imagen, pero necesita más memoria y trabajo de compresión. No se puede deducir de la resolución, por sí sola, que un detector reconocerá todos los objetos.

5.5 Dirección y conexión a la red

La cámara trabaja como cliente de la red creada por la DevKit. Su dirección fija es 192.168.4.2; la puerta de enlace configurada corresponde a la DevKit. La dirección fija permite que la aplicación conozca dónde buscar el flujo sin depender de qué dirección dinámica se entregue en cada arranque.

El servicio dinámico de direcciones de la DevKit reserva otro intervalo para clientes. Si la cámara arranca cuando la red aún no está disponible, la lógica de recuperación realiza intentos de conexión. Es posible encender las placas en orden distinto, pero el video no estará disponible hasta que ambas tengan alimentación y la cámara se haya asociado a la red.

5.6 Un cuadro frente a un flujo continuo

Una captura aislada entrega un cuadro. Un flujo MJPEG es una sucesión de cuadros JPEG que se envían por una misma conexión; cada cuadro se puede mostrar de forma individual. Para la percepción, la edad de cada cuadro importa: si se procesa una imagen vieja, el carro puede haber cambiado de posición desde el momento de captura.

La implementación conserva el cuadro más reciente en una casilla compartida. Cuando el productor genera otro cuadro antes de que se consuma el anterior, el anterior se sustituye. La ventaja es evitar una cola creciente de imágenes antiguas; la consecuencia es que no todos los cuadros capturados llegan necesariamente al consumidor.

El código descarta cuadros cuya edad supera el límite configurado al salir de la cámara. Ese filtro no mide el retraso restante de red, decodificación o presentación en la pantalla. Si se necesita medir el retardo total, debe definirse un método que observe el mismo evento al inicio y al final de la cadena, y registrarse varias muestras en condiciones reproducibles.

5.7 Datos por celdas

En el modo de celdas, la imagen se resume mediante promedios de color de bloques. El paquete incluye dimensiones y datos que permiten reconstruir una cuadrícula de análisis. El teléfono solicita este servicio por UDP; la cámara responde y recuerda al cliente. La transmisión usa el puerto 4210. El modo de celdas puede coexistir con la disponibilidad de JPEG en el servicio de flujo.

Este resumen reduce la cantidad de datos respecto de enviar cada píxel, pero también elimina detalle. No es equivalente a una fotografía completa ni transmite por sí mismo la identidad de un objeto. La percepción por color interpreta las celdas según el piso aprendido y sus umbrales; la comunicación por UDP, a diferencia de un flujo confiable de bytes, no garantiza por sí sola la llegada de cada paquete.

5.8 Leer los resultados del barrido

El barrido histórico comparó combinaciones de resolución, formato, reloj y calidad. Cada combinación se evaluó con 100 cuadros y sin un cliente de video externo. A 20 MHz con RGB565, HQVGA registró 12.4 cuadros por segundo y QVGA 8.8 con la calidad indicada. En ese ensayo, la compresión media informada fue 56 ms para HQVGA y 107 ms para QVGA.

La comparación ayuda a entender la elección del valor predeterminado, pero no representa la velocidad de todas las escenas ni la latencia completa de la app. Las condiciones de luz, la complejidad de imagen, la carga del procesador y la presencia de un cliente pueden cambiar el resultado. La cifra de edad del cuadro reportada en el barrido debe conservar su definición de ensayo; no debe reemplazarse por una afirmación de tiempo hasta pantalla.

5.9 Fallas y recuperación

La lógica de recuperación distingue pérdida de WiFi, falta de respuesta de un servicio y ausencia prolongada de cuadros cuando existe demanda. Para algunas condiciones intenta reconexión y para otras solicita reinicio. El registro de estado ayuda a investigar si hubo reinicios o reconexiones; observar video después de una recuperación no identifica por sí solo la causa inicial.

Una autoprueba local puede verificar que ciertos servicios respondan dentro de la placa, pero no reproduce el enlace inalámbrico con el teléfono. Si la autoprueba pasa y la aplicación no recibe video, aún deben revisarse la red, la dirección, la sesión de video y el cliente conectado. Si el sensor no inicializa, no se debe asumir que cada falla será recuperada automáticamente.

Dónde está en el código: `camara_prueba.ino`, `recuperacion.c`, `paquete_celdas.c`.

6. Protocolo de comunicación

La aplicación y el carro intercambian órdenes y estados por un WebSocket en la red local. El protocolo utiliza mensajes de texto breves; el video de la cámara sigue una vía distinta. Los comandos principales se reúnen en el anexo.

6.1 Movimiento y parada

FORWARD y REVERSE ordenan avance y retroceso; LEFT y RIGHT ordenan pivotes. STOP y EBRAKE detienen los motores. En control manual, la aplicación repite la orden mientras el usuario mantiene pulsado el control y envía STOP al soltarlo. El vigilante detiene los motores cuando el temporizador de comunicación supera el intervalo configurado; el firmware también lo renueva ante algunos mensajes que no son órdenes de movimiento.

PING y PONG permiten comprobar el enlace, pero PING no renueva el plazo de movimiento. La cola protege las órdenes de parada frente a saturación. GIRAR solicita un giro por ángulo y devuelve un resultado como GIRO_OK, GIRO_TIMEOUT o GIRO_ERROR.

6.2 Estado y configuración

MODO_AUTO y MODO_MANUAL seleccionan perfiles; sus respuestas confirman la selección. Los mensajes SENS comunican ángulo, distancia y banderas de estado. TOF_PARO informa una detención activada por el láser. Las consultas de sensores y balance permiten comprobar las condiciones antes de una ruta.

6.3 Interpretación de telemetría

La trama SENS contiene tres campos separados por comas: ángulo en décimas de grado, distancia en milímetros y un entero de banderas. Por ejemplo, un ángulo de 450 representa 45 grados. Las banderas indican el estado del giroscopio, del láser y de otras condiciones del firmware; se deben interpretar junto con la versión del protocolo correspondiente.

El perfil AUTO puede emitir telemetría periódica mientras hay un cliente conectado. Una lectura inválida o atrasada no debe tratarse como una distancia segura. Las respuestas de error o de tiempo agotado de un giro indican que la operación no terminó normalmente y requieren detenerse y revisar las condiciones antes de continuar.

6.4 Direcciones y sesiones

La DevKit crea un punto de acceso local con la dirección 192.168.4.1. La aplicación abre el canal de órdenes en `ws://192.168.4.1/ws`. La cámara se conecta a esa red con la dirección 192.168.4.2 y ofrece su video por HTTP. El teléfono se incorpora como cliente de la misma red y puede hablar con ambos nodos.

Una dirección IP identifica un dispositivo dentro de la red; una ruta como `/ws` identifica el servicio solicitado en ese dispositivo. La DevKit asigna direcciones de cliente mediante DHCP en un intervalo que deja libre la dirección fija de la cámara. Si otro equipo usa una dirección fija que coincide con la de la cámara, el resultado puede ser una conexión ambigua; la dirección debe revisarse en la configuración de ese equipo.

El firmware mantiene un cliente WebSocket activo a la vez. Si otro cliente abre el control, puede cerrar la sesión anterior. Por esa razón no conviene controlar simultáneamente desde el navegador y desde la aplicación. La sesión de video de la cámara también tiene una limitación de clientes que debe considerarse por separado.

6.5 Orden mantenida y pulso de conexión

En conducción manual, mantener pulsado un control produce repeticiones de la orden de movimiento. Al soltarlo, la aplicación envía STOP. PING y PONG son mensajes de sondeo: ayudan a saber si hay respuesta, pero PING no renueva el temporizador que protege el movimiento. La app y el firmware realizan tareas distintas, por lo que un PONG recibido no demuestra que una orden de motor se haya ejecutado ni que el sensor vea el camino libre.

El temporizador se revisa cuando hay motores en marcha y no está activo un giro por ángulo. El valor configurado es 1000 ms y la condición de código usa una comparación de tiempo transcurrido. El firmware renueva el contador con algunos textos que no son movimiento; incluso un texto desconocido puede renovarlo. Esto limita la interpretación de la protección: no se debe afirmar que solo una orden de movimiento válida mantiene vivo el temporizador.

Los giros por ángulo tienen su propio plazo y su respuesta. Si se interrumpen con una parada o aparece una falla del giroscopio, la aplicación debe tratar la respuesta como un resultado de la maniobra y no como una confirmación de la ruta completa. El tiempo hasta que la carrocería deja de moverse también incluye la inercia después de apagar la señal PWM.

6.6 Interpretar un ejemplo de sensores

En el mensaje `SENS:450,320,7`, el primer campo representa 45.0 grados y el segundo 320 mm. El tercero es una combinación de banderas: 7 equivale a 1 + 2 + 4, que en esta versión indica las condiciones definidas para giroscopio listo, sensor de distancia presente y distancia válida. Una bandera informa lo que el firmware declaró en ese instante; no prueba que una lectura permanezca vigente después ni que la orientación física esté calibrada contra una referencia externa.

El valor 320 mm es una distancia del sensor frontal, no el ancho de una caja ni la distancia a cualquier punto alrededor del carro. Si el objeto no intercepta el haz, o la medición es inválida, la aplicación necesita interpretar esa condición junto con la percepción visual y el estado del sensor. Un valor de respuesta debe acompañarse de su unidad y condición de validez al registrarlo.

6.7 Consultas, confirmaciones y errores

CAPS? consulta la disponibilidad declarada de sensores. IMU? permite solicitar información del giroscopio y TOF? del sensor de distancia. IMU_CAL pide recalcular el sesgo en reposo y puede rechazarse si no se cumplen sus condiciones; IMU_CERO establece una referencia angular. TOF_USO permite habilitar o deshabilitar la autoridad de parada del láser. Un cambio de configuración debe confirmarse mediante la respuesta del carro.

Los perfiles se solicitan con MODO_AUTO y MODO_MANUAL y devuelven confirmaciones distintas. Los factores de balance se pueden consultar y modificar por perfil dentro del intervalo permitido por el firmware. La memoria no volátil puede conservar un valor previo; por eso un dato predeterminado leído en el código no basta para afirmar el valor efectivo de un dispositivo sin consultarlo.

LOOP? informa el máximo registrado del tiempo de una vuelta y el número de excesos respecto al umbral configurado. Un máximo y un contador sirven para detectar que ocurrió al menos un exceso; no muestran por sí mismos la causa ni el tiempo medio de todas las vueltas. La interpretación requiere anotar cuándo se reinició el contador y qué actividad se realizó durante la ventana observada.

Dónde está en el código: `carro.c`, `sensores.c`, `ControlLogica.kt`, `Enlace.kt`.

7. Núcleo de navegación en C

El núcleo transforma una ruta dibujada en la aplicación en acciones de avance y giro. Su implementación en C también puede ejecutarse en una computadora para probar la lógica con un vehículo simulado.

7.1 Ruta, calibración y pose

La cuadrícula de la aplicación representa celdas de 20 cm. La trayectoria se descompone en tramos y cambios de dirección. Como el carro no dispone de encoders para medir el giro de las ruedas, la distancia de avance se estima a partir del tiempo y de una calibración en milisegundos por centímetro. La calibración también contempla el arranque, el frenado y los giros.

La pose es la posición y orientación estimadas. Durante un avance, el navegador integra incrementos de distancia con el rumbo observado por el giroscopio cuando la lectura es válida. El giro medido mejora la orientación estimada, pero no detecta un deslizamiento lateral sin rotación. Los errores de calibración y de tracción pueden acumularse.

7.2 Esquive y reincorporación

Ante un obstáculo, el navegador puede detenerse, evaluar una alternativa lateral y planear el regreso a la trayectoria. Si no puede continuar, puede detenerse o solicitar ayuda según la configuración. La corrección J15 integra el rumbo observado durante el avance para actualizar la pose que usa la reincorporación.

En el escenario B6 **SIMULADO**, el error lateral medio pasó de 13.2 a 1.9 cm después de la corrección. Se terminaron 400 de 400 rutas y se registraron cero contactos en ambas versiones; con error menor de 20 cm terminaron 344 y 400, respectivamente. La validación de esa corrección en el carro físico permanece **NO MEDIDA**.

7.3 Simulación

El simulador ejecuta la lógica del navegador sobre un modelo de movimiento y obstáculos. Permite repetir escenarios con condiciones controladas, pero no reproduce toda la percepción real, el deslizamiento, la iluminación ni la red. Un resultado simulado no demuestra que el vehículo físico siga la misma trayectoria.

7.4 Ajuste de movimiento

La calibración relaciona una distancia conocida con el tiempo de avance observado y contempla el tiempo que tarda el carro en comenzar a desplazarse. El frenado representa la distancia recorrida después de ordenar la parada. Esos parámetros son específicos del montaje y de las condiciones del ensayo: una modificación de peso, pilas o superficie puede cambiar su efecto.

Los giros izquierdo y derecho no son necesariamente simétricos. El firmware utiliza el giroscopio para observar el ángulo y puede anticipar la detención para reducir el sobrepaso. Si el sensor no está listo o se agota el tiempo, la respuesta de giro informa el fallo. Una lectura tardía no debe presentarse como una posición absoluta medida.

7.5 Lectura de la simulación

El simulador distingue la posición de su vehículo virtual de la posición estimada por el navegador. En el ensayo B6 la caja, el chasis, el retraso de video y la variación de giro forman parte del modelo; algunas de esas magnitudes son supuestos. Se registran llegadas, contactos y error lateral. Esas métricas no miden la precisión longitudinal ni demuestran que una ruta física quede libre de contacto.

7.6 De una cuadrícula a movimientos

La ruta se describe mediante puntos de una cuadrícula. Una celda representa 20 cm, de modo que dos puntos separados por cinco celdas en una misma dirección representan un tramo nominal de 100 cm. El núcleo transforma cada cambio de dirección en un giro y cada tramo en un avance. La duración del avance depende de la calibración; la cantidad de celdas no indica cuánto recorrieron realmente las ruedas.

Por ejemplo, si una calibración hipotética indica 50 ms por centímetro, 100 cm corresponden a 5000 ms de movimiento calculado antes de considerar el tratamiento de arranque y frenado. La cifra es un ejemplo de cálculo, no una medición nueva. El programa contempla parámetros adicionales para el tiempo inicial y el movimiento posterior a la parada; esos valores deben corresponder al carro y al entorno utilizados.

Una ruta dibujada y una ruta ejecutada son objetos distintos. La primera contiene la intención del operador; la segunda resulta de las decisiones del navegador, de las observaciones disponibles y de cómo responde el vehículo. Si se cambia la ruta o se interrumpe la ejecución, debe comprobarse el estado de parada antes de volver a iniciar.

7.7 Estados y transiciones

El navegador organiza su trabajo como una máquina de estados: en cada instante tiene un estado y aplica las reglas asociadas a él. En ESPERANDO existe una ruta cargada que aún no se ejecuta. En SIGUIENDO se procesa el paso actual. En DETENIDO_OBSTACULO se mantiene la parada mientras se decide si es posible continuar o esquivar.

En ESQUIVANDO se ejecutan las maniobras para apartarse; en REINCORPORANDO se calcula el regreso a la línea original. TERMINADO indica fin de ruta y ERROR indica que no pudo continuarse. En ESPERANDO, TERMINADO y ERROR el núcleo devuelve STOP. La opción AYUDA permite intervención de una persona cuando está habilitada; durante esa intervención el responsable debe supervisar el espacio y las órdenes manuales.

El estado es una descripción de la lógica, no una medición independiente del movimiento. Por ejemplo, TERMINADO indica que se alcanzó el criterio de la ruta en el modelo del navegador; no certifica por sí solo la posición física del chasis. La posición real se compara con una referencia externa, como una marca y una cinta métrica, cuando se desea medir error.

7.8 Confirmación de obstáculos y pérdida de imagen

La cercanía visual se expresa por niveles; no es una distancia física directa en centímetros. El código trata un obstáculo muy cercano con mayor urgencia que uno lejano. Para una detección de nivel 2 en una zona lateral se requiere confirmación temporal: 150 ms y al menos dos cuadros. Para volver a avanzar se exige una condición de frente libre durante 250 ms. Estos son parámetros del software, no resultados de tiempo de reacción medidos en el conjunto físico.

La ausencia de cuadros también se considera una condición de riesgo. Para una ruta que exige cámara, el núcleo define un hueco máximo de video de 700 ms y, al superarlo, detiene la ruta y pasa a ERROR. Al empezar existe además un plazo para recibir el primer cuadro mientras el carro permanece quieto. La aplicación tiene sus propias comprobaciones; el valor de una capa no debe confundirse con una medición de latencia de toda la cadena.

7.9 Desvío y regreso a la ruta

La configuración plantea un giro de 90 grados para apartarse y un tramo lateral nominal de 40 cm. El navegador busca superar el obstáculo antes de volver hacia la línea; el punto de reincorporación se define 30 cm por delante del punto de salida. Esos valores describen el plan del algoritmo, no las distancias necesariamente alcanzadas por el carro físico.

El regreso se calcula a partir de la pose estimada. Si el carro ha girado o patinado de una manera que el modelo no representa, el punto calculado puede diferir del lugar real. La Figura 7 ilustra una trayectoria conceptual y no representa un plano de montaje ni una medición del recorrido.

El código también limita el retroceso de los reintentos: 15 cm y un máximo de 2000 ms. El vehículo no cuenta con un sensor posterior en esa maniobra. Por eso el operador debe mantener libre la zona detrás del carro; el límite programado no detecta a una persona, una pared o un borde en esa dirección.

7.10 Datos que conserva la calibración

El modelo de calibración incluye el tiempo de arranque, el tiempo por centímetro, el avance posterior al STOP y parámetros separados para giros izquierdo y derecho. Puede incluir retroceso y desplazamiento del centro del carro durante un pivote. Separar ambos lados permite representar la asimetría del chasis, en vez de asumir que los dos motores y las dos maniobras son idénticos.

La aplicación guía mediciones por partes. En la prueba de avance se registran distancias recorridas para distintos tiempos y repeticiones; el cálculo ajusta los parámetros a esos datos. El tiempo de arranque incluye una intervención humana cuya reacción debe

considerarse. Si se usan valores de ejemplo o parámetros antiguos, el simulador y la ruta real pueden divergir aunque ambos ejecuten correctamente el mismo código.

La guía histórica de calibración afirma que el carro no tenía IMU. Esa descripción corresponde a una etapa anterior: el código actual utiliza el MPU6050. El avance continúa estimándose por tiempo, mientras que el giroscopio aporta información de giro y rumbo. Ambas afirmaciones pueden coexistir cuando se indica qué magnitud se mide y cuál se estima.

La Figura 6 presenta: Ciclo general de navegación y estimación de pose.

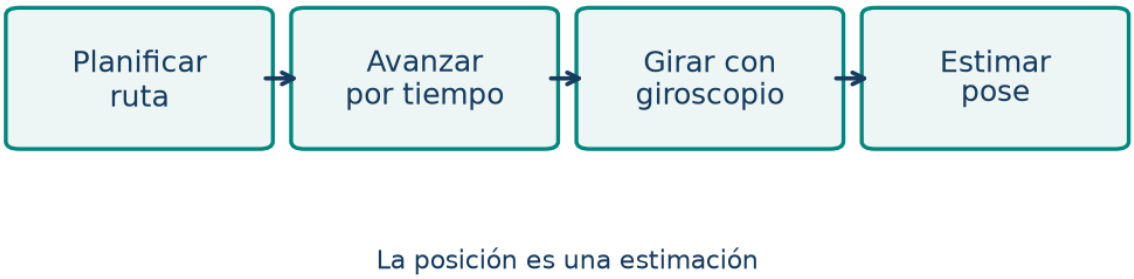


Figura 6. Ciclo general de navegación y estimación de pose.

La Figura 7 presenta: Etapas conceptuales del esquive y la reincorporación.

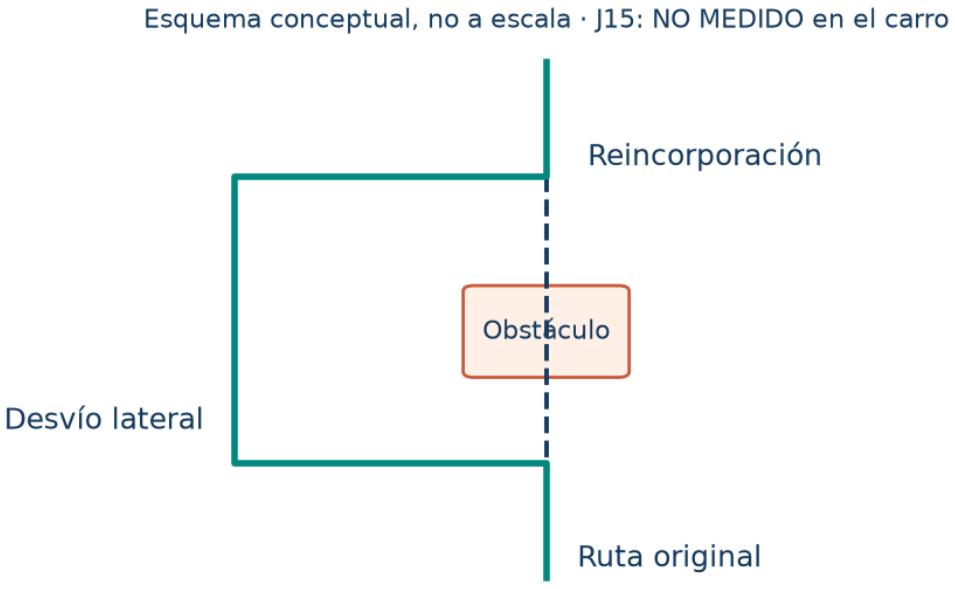


Figura 7. Etapas conceptuales del esquive y la reincorporación.

Dónde está en el código: `trayectoria.c`, `calibracion.c`, `navegador.c`, `simulador_navegador.c`, `sim_ruta.c`.

8. Visión y percepción

La percepción ayuda a reconocer obstáculos que pueden interferir con una ruta. El sistema combina señales de color y de un modelo de objetos con información de distancia frontal; cada fuente tiene limitaciones propias.

8.1 Color, objetos y corredor

El detector de color aprende una referencia del piso y analiza celdas de la imagen que se apartan de ella. Sombras, reflejos o cambios de superficie pueden afectar el resultado. El modelo de objetos usa una red neuronal para identificar objetos en imágenes; puede omitirlos o generar detecciones erróneas, por lo que no sustituye la comprobación del entorno.

El corredor es la zona proyectada de la imagen por la que se espera que pase el carro. Su forma depende de la perspectiva y de los parámetros de montaje. Permite distinguir obstáculos que invaden el camino de los que aparecen a un lado. La regla básica de fusión conserva la señal de seguridad del detector de color; otras etapas opcionales pueden modificar el resultado.

8.2 Distancia frontal

El láser VL53L0X aporta una medición de distancia en la dirección a la que apunta. Su campo es limitado: no identifica por sí solo todos los objetos laterales, desniveles o huecos. Cuando su autoridad está activa y se cumplen las condiciones del firmware, puede ordenar una parada durante el avance en AUTO. La combinación busca aprovechar señales distintas, pero no garantiza detectar todos los obstáculos.

8.3 De la imagen a una decisión

La referencia de piso depende de la apariencia de la superficie visible. El detector compara colores por celdas y agrupa los resultados en zonas, con confirmación temporal para evitar reaccionar a una sola observación aislada. Un objeto puede ser difícil de distinguir si su color se parece al piso; una sombra o un reflejo puede parecer un objeto. Por eso la preparación debe hacerse en el lugar de uso y con el área despejada.

El corredor representa una franja de paso proyectada por perspectiva: se ve más ancha cerca de la cámara y más estrecha hacia el horizonte. Su posición depende de la geometría y la inclinación de la cámara. Mover la cámara sin revisar esos parámetros puede hacer que una detección se interprete fuera de la zona prevista.

La regla básica de fusión da prioridad a la señal de seguridad del detector de color y permite que el modelo incremente la severidad. El láser actúa por una vía de parada separada bajo las condiciones configuradas. Las fuentes no tienen el mismo campo de visión ni la misma latencia; una coincidencia entre ellas no debe suponerse en todas las situaciones.

El aprendizaje del piso admite varios grupos de color y puede actualizarse cuando la apariencia cambia. El proyecto documenta un riesgo: una pared que llena toda la

imagen, incluida la franja de referencia, puede confundirse con un cambio completo de piso y aprenderse como tal. Esta limitación exige mantener supervisión y no confiar únicamente en la clasificación por color.

8.4 Formación de la cuadrícula de color

El detector divide la imagen en bloques de 10 por 10 píxeles y calcula un color promedio para cada bloque o celda. Si la imagen tiene 320 por 240 píxeles, la cuadrícula tiene 32 columnas y 24 filas. Si tiene 240 por 176, se utilizan 24 columnas y 17 filas completas; los píxeles que no completan un bloque no forman una celda adicional.

El promedio reduce el efecto de variaciones aisladas, pero también elimina detalle. Un objeto pequeño que ocupa pocas celdas puede no alcanzar el mínimo configurado para marcar una zona. Al reducir la resolución, cada celda representa una parte mayor de la escena; por ello una imagen más pequeña no equivale a la misma capacidad de detección con menos datos.

Una franja de la parte inferior y central aporta la referencia inicial de piso. El algoritmo compara el color de las demás celdas con esa referencia. Si la franja está ocupada por un objeto o no representa la superficie que se recorrerá, la referencia puede ser inadecuada. Preparar el área despejada es una condición del método, no una calibración opcional de presentación.

8.5 Color, brillo y varios tonos

La comparación considera tanto diferencias directas de los canales rojo, verde y azul como una comparación de tono que intenta tolerar ciertos cambios de iluminación. El brillo indica cuánta intensidad total tiene el color; el tono normalizado permite comparar colores parecidos aunque uno se vea más claro. En zonas demasiado oscuras el tono puede ser poco confiable.

El piso no siempre es uniforme. Las baldosas, vetas y uniones producen colores distintos incluso cuando la superficie está libre. El modelo admite varios grupos de color para representar esa variación. Durante el aprendizaje continuo se restringe qué celdas pueden modificar la referencia, con el propósito de no incorporar como piso las celdas ya consideradas obstáculo o sus vecinas.

La restricción reduce algunos errores, pero no distingue de forma infalible un cambio de piso de una superficie que ocupa toda la imagen. Si una pared cubre también la franja de referencia, el sistema puede acabar interpretándola como un piso nuevo. La supervisión humana y la parada física siguen siendo necesarias incluso cuando el algoritmo informe una zona libre.

8.6 Zonas y niveles de cercanía

El resultado de color resume información por zonas izquierda, centro y derecha. La cercanía va de 0 a 3 y se relaciona con la posición de las celdas detectadas dentro de la imagen. Un nivel más alto señala una observación que el algoritmo considera más próxima, pero no equivale a una distancia medida por el láser.

La perspectiva explica parte de la regla: una región próxima al vehículo suele aparecer más abajo en la imagen. No obstante, la relación depende de la altura y orientación de la

cámara y de la forma del objeto. Un objeto alto o una imagen recortada pueden alterar la interpretación. Para determinar una distancia física se necesita una medición apropiada y su condición de validez.

Un obstáculo de nivel 1 puede producir solo un aviso, según la configuración del navegador. Los niveles 2 y 3 participan en reglas de detención y confirmación. La aplicación debe distinguir el dato visual de la orden final de movimiento: una zona marcada no es, por sí sola, una prueba de que el firmware haya detenido los motores.

8.7 Modelo de objetos y vigencia

El modelo de objetos procesa imágenes en un hilo separado del detector de color. El hilo recibe el cuadro más reciente disponible y publica detecciones con una marca de tiempo. Esto evita que el detector de color espere cada inferencia, pero significa que los dos resultados pueden corresponder a momentos distintos.

La configuración de fusión descarta un resultado del modelo demasiado antiguo; el valor de referencia documentado es 300 ms. La edad se calcula respecto a la marca del cuadro, no solo desde que terminó el cálculo. Si el modelo no está disponible o su resultado caduca, la regla básica continúa utilizando el detector de color. La vigencia configurada es una regla de software, no una medición de tiempo de reacción completo del carro.

El modelo entrega cajas que delimitan regiones de la imagen y una confianza para sus detecciones. La posición de la caja y de su borde inferior ayuda a asignar zona y cercanía visual. Una confianza es una puntuación del modelo bajo sus reglas de inferencia; no es una garantía física de que el espacio esté libre o de que se reconozcan todas las clases.

8.8 Reglas básicas de fusión

Cuando el detector de color ya señala un obstáculo cercano, la regla básica conserva esa señal sin esperar una confirmación del modelo. Si el modelo ve un objeto en el camino y el detector tiene una señal menos concluyente, la fusión puede elevar la severidad. Si el modelo detecta algo lejano que el detector no ve, puede producir un aviso. Para un objeto cercano visto solo por el modelo se requiere una confianza mayor para ordenar una parada desde esa señal.

Estas reglas se ejecutan sobre datos y umbrales configurados, y existen otras etapas opcionales de percepción que pueden modificar el resultado. La autoridad del láser corresponde a una protección distinta en el firmware. Ninguna de estas capas proporciona cobertura completa del entorno ni permite abandonar la supervisión del operador.

La Figura 8 presenta: Señales de percepción y parada frontal condicionada.

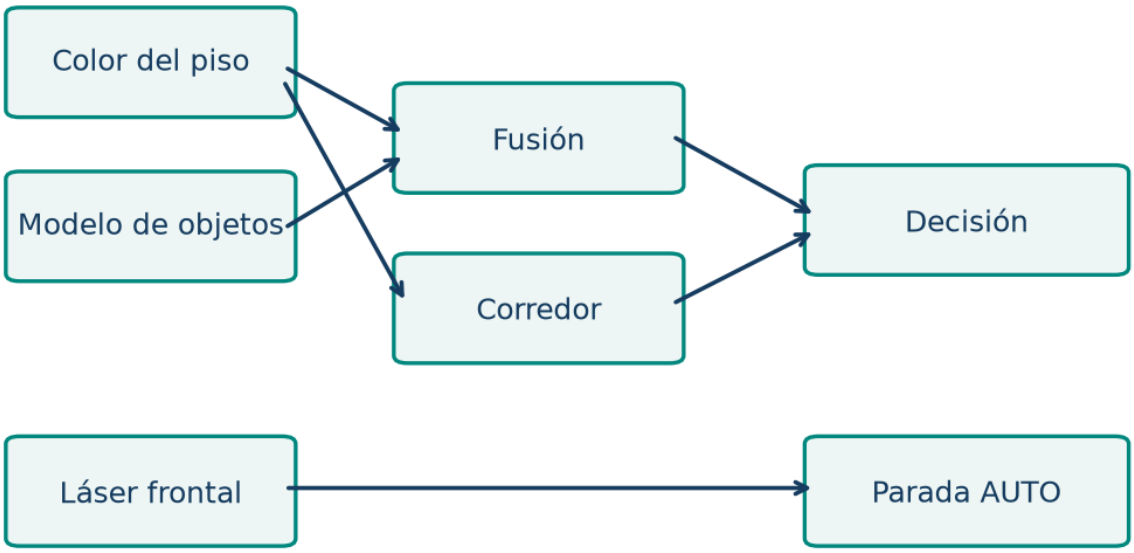


Figura 8. Señales de percepción y parada frontal condicionada.

Dónde está en el código: `obstaculos.c`, `fusion.c`, `corredor.c`, `DetectorTflite.kt`, `tof.c`.

9. Aplicación Android

La aplicación reúne los controles de conducción, el editor de rutas y las pantallas de diagnóstico. Se comunica con el carro y con la cámara por la red local y llama al núcleo de navegación en C mediante una interfaz nativa.

9.1 Pantallas principales

Inicio presenta el estado de conexión y accesos rápidos. Manual reúne los controles de dirección y parada. Rutas permite dibujar, simular y ejecutar trayectorias. Más da acceso a ajustes y herramientas como sensores y calibración. Las sesiones de conexión y video se comparten entre pantallas para evitar abrir un cliente independiente en cada una.

El modo manual envía órdenes mientras se mantiene pulsado un control y solicita parada al soltarlo. En una ruta, la aplicación suministra al navegador las observaciones y transmite las órdenes que este decide. Si se pierde la conexión o dejan de llegar cuadros durante el tiempo previsto, la ruta se cancela según la lógica de la aplicación.

9.2 Uso del video y percepción

La cámara del carro y la cámara del teléfono son fuentes distintas. Algunas pantallas usan el video de la Freenove; la pantalla de visión puede usar la cámara del teléfono. Conviene comprobar cuál fuente está seleccionada antes de interpretar una detección. La cámara del carro atiende un cliente de video a la vez, y el firmware atiende un cliente WebSocket de control a la vez.

9.3 Preparación de una ruta

En Rutas se dibujan los puntos de paso sobre la cuadrícula. La simulación muestra el resultado del modelo y permite revisar la secuencia antes de poner el carro en el piso. La ejecución real depende de la calibración guardada, de los sensores, de la imagen y de la conexión. Si la aplicación no recibe confirmación del perfil AUTO, no debe interpretarse la ruta como iniciada correctamente.

La pantalla Sensores permite consultar el estado del giroscopio y del láser, y la de calibración de giro guía las pruebas que alimentan sus parámetros. Esos procedimientos se realizan con el carro colocado y el área despejada. Cambiar de pantalla mientras hay una ruta en marcha requiere atender la indicación de la aplicación y comprobar que la ejecución se haya detenido.

9.4 Organización de la interfaz

Inicio reúne el estado general. Desde ahí se puede comprobar si el carro y la cámara responden, y revisar indicadores disponibles antes de conducir. Un indicador de conexión describe el estado de una sesión de red; no confirma automáticamente que las ruedas estén libres o que el área sea segura.

Manual presenta los controles de movimiento, la selección de perfil y el acceso a parada. En esta pantalla el operador mantiene una orden mediante el control táctil. La repetición de mensajes es una parte del diseño de comunicación, pero el operador debe verificar visualmente que el vehículo responda como se espera.

Rutas contiene el editor y la ejecución. El editor representa el espacio mediante celdas y permite definir puntos. La simulación utiliza un modelo para representar el recorrido antes de ejecutarlo; debe interpretarse como una ayuda de planeación. Más reúne opciones de configuración y herramientas adicionales, incluidas pantallas de calibración y sensores.

9.5 Conexión compartida

La aplicación mantiene una sesión compartida del carro y otra del video. Las pantallas que usan esos recursos se incorporan a la sesión existente en lugar de abrir cada una su propia conexión. Al salir de una pantalla hay un periodo breve de espera antes de cerrar el recurso cuando ya no lo usa ninguna otra. Este diseño reduce cortes al cambiar entre pantallas o cambiar la orientación del teléfono.

La sesión compartida no elimina la restricción del firmware de un cliente de control. Si un navegador u otra aplicación se conecta al mismo servicio, la conexión anterior puede terminar. Una pantalla que muestra reconexión indica que la aplicación está intentando recuperar el enlace; debe esperarse a confirmar el estado antes de ordenar movimiento.

9.6 Conexión y selección de perfil

El módulo de control abre el WebSocket en la dirección configurada. Envía PING periódicamente y utiliza PONG para observar la respuesta del enlace. Mientras se mantiene presionado un control manual repite la orden de movimiento; al soltarlo envía STOP. La configuración del proyecto usa 800 ms para el sondeo y 250 ms para la repetición de movimiento, valores que no equivalen a una medición de latencia total.

La aplicación solicita un perfil y espera la confirmación correspondiente. Si una interrupción provoca que el firmware regrese a AUTO, la aplicación vuelve a solicitar el perfil deseado. No debe suponerse que la selección visible fue aceptada si falta la confirmación. Para iniciar una ruta se exige el perfil AUTO; una falla de comunicación puede impedir el inicio o cancelar la ejecución.

9.7 Trabajo entre Android y el núcleo

La interfaz está escrita en Kotlin y el núcleo de navegación en C. JNI es el mecanismo que permite que la aplicación llame funciones nativas y reciba resultados. El puente transforma datos entre ambos lados; la lógica del navegador se conserva en el núcleo, que también puede compilarse en una computadora para pruebas.

El ciclo semiautónomo reúne la ruta, la calibración y el resultado disponible de percepción, llama al navegador y transmite los comandos que corresponden. La separación es útil para ensayar el algoritmo sin teléfono, pero la aplicación real añade sus propias condiciones de sesión, pantalla, conexión y cámara. Una prueba del núcleo no cubre automáticamente esas condiciones.

9.8 Estados de las capturas

Las capturas muestran momentos distintos. La vista conectada ilustra un enlace activo; la vista de reconexión contiene un aviso de que el teléfono no está en la red del carro; la de ajustes muestra una sesión sin conexión. Son ejemplos de interfaz tomados en circunstancias diferentes y no una secuencia que demuestre una ruta ejecutada.

Cuando el control está disponible y falla solo el video, el canal de parada puede seguir funcionando porque es independiente. Sin embargo, no se debe iniciar o continuar una ruta que requiere percepción si no llegan cuadros confiables. En todos los casos se conserva una forma física de cortar la alimentación de los motores.

La Figura 9 presenta: Pantallas principales de la aplicación.

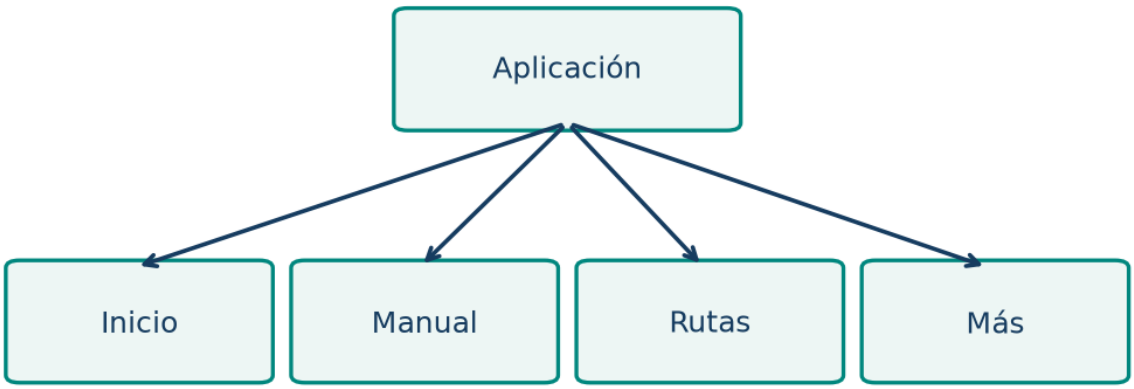


Figura 9. Pantallas principales de la aplicación.

La Figura 10 muestra las vistas correspondientes.



Conectado



Reconectando: teléfono fuera de la red



Sin conexión

Figura 10. Pantallas capturadas en distintos estados de conexión.

Dónde está en el código: `MainActivity.kt`, `SesionCarro.kt`, `SesionVideo.kt`, `ModoSemiautonomo.kt`.

10. Compilación, instalación y versión

El proyecto tiene productos distintos: firmware para la DevKit, firmware para la cámara, núcleo de pruebas para computadora y aplicación Android. Este capítulo indica cómo identificarlos y dónde consultar los procedimientos de compilación e instalación.

La compilación de las placas utiliza Arduino CLI y el soporte ESP32; la aplicación utiliza Gradle, Android SDK, NDK y CMake. El núcleo se compila con un compilador C y make. Las versiones y dependencias deben verificarse en los archivos de configuración antes de reproducir una compilación.

Desde la carpeta de la aplicación, `gradlew.bat assembleDebug` genera una APK de depuración. El script de carga permite compilar el firmware sin escribirlo en una placa mediante la opción `-SoloCompilar`. La instalación o carga sobre dispositivos físicos requiere preparar el equipo y verificar el puerto antes de ejecutarla.

El verificador de versiones compara las identificaciones disponibles con el repositorio y el artefacto. Una identificación de archivo o un resultado histórico no prueban qué versión está instalada actualmente en un dispositivo sin realizar la lectura correspondiente. El reporte J15 verificó la APK instalada en esa sesión; la lectura actual del firmware quedó pendiente en ese cierre.

10.1 Requisitos registrados

La configuración Android declara SDK de compilación 37, herramientas de compilación 36.0.0, NDK 30.0.16248370 y CMake 4.1.2. El registro de compilación de las placas utiliza el núcleo ESP32 3.3.12. Los requisitos deben mantenerse acordes con los archivos de configuración de la revisión que se compile.

10.2 Compilación y pruebas sin carga

Desde la raíz del repositorio, `./scripts/cargar.ps1 nuevo -SoloCompilar` compila el firmware del carro sin cargarlo. Para la cámara, `./scripts/cargar.ps1 camara -SoloCompilar` realiza la operación equivalente. En las carpetas del núcleo o de pruebas de firmware, `make test` ejecuta las suites correspondientes cuando están disponibles el compilador y make.

La opción `--solo-apk` del verificador limita la comprobación a la aplicación y deja pendiente la lectura del firmware. Cuando se dispone de acceso autorizado a los dispositivos, conviene registrar las versiones leídas y el resultado de la comparación. Los binarios con credenciales de red no deben publicarse.

10.3 Identificar el producto antes de actuar

Una compilación transforma fuentes en un artefacto para una plataforma determinada. La aplicación de Android, el firmware del carro y el firmware de la cámara no son intercambiables: cada uno tiene dependencias, funciones y destino propios. El núcleo C permite ejecutar lógica en computadora y también se integra con la aplicación; una prueba de ese núcleo no carga ninguna placa.

Antes de iniciar, registre la revisión del repositorio y el resultado de `git status`. Si hay cambios locales, anótelos para saber si el resultado corresponde exactamente a un commit o a una mezcla con archivos modificados. No es necesario publicar archivos locales de configuración para conservar esa trazabilidad.

10.4 Diferencia entre compilar y cargar

Compilar verifica que las herramientas puedan construir un programa con la configuración disponible. Cargar escribe firmware en una placa, por lo que sustituye el programa existente. La opción `-SoloCompilar` detiene el script antes de la identificación y carga de la placa; es apropiada para comprobar la construcción cuando no se pretende modificar hardware.

El script distingue los modos `baseline`, `nuevo` y `camara`. El modo nuevo usa la carpeta del firmware del carro y requiere la configuración local de secretos. El modo cámara corresponde a la placa con cámara. Es necesario seleccionar el modo que coincide con el dispositivo y comprobar el puerto antes de cargar.

Hay una particularidad importante: en los modos de identificación automática `baseline` y `camara`, una placa no reconocida puede recibir una sonda para identificarla; esa sonda sobrescribe el programa existente. El propio script describe que indicar un puerto conocido evita esa detección automática. Antes de una carga, conserve los respaldos necesarios y compruebe la identidad de la placa; no trate la detección como una operación siempre de solo lectura.

10.5 Interpretación de errores de construcción

Un fallo por herramienta ausente debe resolverse instalando o configurando la dependencia indicada por el proyecto. Un fallo al descargar dependencias puede relacionarse con acceso de red o repositorios. Un error de compilación, en cambio, señala un problema al procesar fuentes o configuración y requiere leer el primer diagnóstico pertinente, no solo la última línea de salida.

Una construcción exitosa demuestra que se produjo un artefacto para esa configuración. No demuestra que se haya instalado, que la placa sea la correcta ni que la aplicación se comporte correctamente en el teléfono. Después de una instalación autorizada hay que comprobar la versión en el dispositivo y ejecutar pruebas apropiadas al cambio.

10.6 Manejo de configuración sensible

Las credenciales pertenecen a la configuración local. Al compartir un registro, revise tanto la salida como los archivos adjuntos: una captura o un binario también pueden contener información sensible. El script intenta ocultar contraseñas conocidas en su salida, pero esa protección no sustituye la revisión de lo que se comparte.

Una dirección de red identifica un servicio dentro de la configuración descrita; no es una prueba de identidad del firmware. Del mismo modo, un nombre de archivo que contiene una versión no demuestra que el archivo se encuentre instalado en una placa. Para la trazabilidad se registran por separado revisión fuente, artefacto generado, dispositivo y lectura de versión disponible.

Dónde está en el código: `cargar.ps1`, `verificar_version.py`, `build.gradle.kts`, `Makefile`.

11. Operación y seguridad física

El vehículo requiere supervisión constante y un espacio despejado. Antes de moverlo se comprueban la alimentación, la conexión, los sensores y la disponibilidad de una parada accesible.

11.1 Preparación y conexión

Con los motores apagados, comprobar la fijación de cables, ruedas y sensores. Conectar cada placa a su propia batería portátil y encender la DevKit para que aparezca la red CarroFSI. Conectar el teléfono a esa red y abrir la aplicación. Verificar en Inicio el enlace con el carro y la cámara; consultar la pantalla de sensores con el carro quieto.

Antes de habilitar los motores, despejar la pista y comprobar que el operador puede accionar PARO y el interruptor físico del portapilas. Para una ruta, colocar el carro en la posición inicial, comprobar el rumbo y la distancia frontal, y revisar la calibración y la fuente de imagen. Simular una ruta ayuda a revisar su trazado, pero no sustituye una prueba física.

11.2 Durante la operación

En Manual se mantiene pulsado el control de dirección y se suelta para detener. En Rutas se selecciona la trayectoria y se inicia con el área libre. Mantener una persona cerca de la parada física; si el carro no responde a la aplicación, apagar la alimentación de los motores.

El proyecto emplea como regla de ensayo un máximo de 20 corridas o 90 minutos con el mismo juego de pilas. Esa regla de operación no es una medición de autonomía. Cambios de pilas, superficie y agarre pueden invalidar una calibración previa.

11.3 Preparar el entorno

La pista debe permitir observar el carro durante toda la maniobra y alcanzar el interruptor de motores. Se retiran objetos que puedan atrapar una rueda y se mantienen personas, cables y bordes fuera de la trayectoria. En una prueba de retroceso se deja también espacio libre detrás: el carro no mide esa zona.

Para comparar recorridos se marca un punto inicial y una línea de referencia. Una cinta métrica permite medir distancias y desvíos en el piso. La posición inicial debe ser repetible: las ruedas apoyadas, el frente alineado y la ubicación anotada. Si cambia la posición de la cámara, la iluminación o la superficie, se registra el cambio antes de comparar resultados.

Antes de energizar, se revisa que las baterías estén en condiciones adecuadas y que el portapilas de motores esté apagado. Se comprueba que cables y conectores no estén sueltos ni rocen las ruedas. Cada placa usa su propia powerbank; las cuatro pilas AA alimentan el driver de motores. Ninguna de estas comprobaciones determina por sí sola la autonomía restante.

11.4 Encender y verificar sin movimiento

Conectar la batería de la DevKit y la de la cámara. La DevKit crea la red; la cámara debe asociarse a ella. En el teléfono, seleccionar CarroFSI desde los ajustes de red y abrir la aplicación. En Inicio se verifica el enlace con el carro y la disponibilidad de la cámara. Si no aparece la red, se revisan alimentación y cable antes de buscar fallas en la aplicación.

Con el carro inmóvil se consulta Sensores. Se revisa que el giroscopio esté listo y que la lectura de distancia sea coherente con el espacio frontal. Si el rumbo cambia con el carro quieto, puede ser necesario repetir la calibración de sesgo siguiendo la indicación de la interfaz. La referencia de cero debe establecerse con el carro en la orientación que se utilizará como inicio.

Si el video no llega, comprobar la alimentación de la cámara, la red, la dirección configurada y si otro cliente está ocupando la sesión. No se inicia una ruta que requiera cámara mientras no se reciban cuadros. El botón de parada y el interruptor físico deben estar accesibles antes de energizar los motores.

11.5 Conducir en manual

En la pantalla Manual se selecciona el perfil deseado y se verifica su confirmación. Antes de pulsar una dirección se revisa el espacio correspondiente. El control se mantiene pulsado mientras se desea sostener la orden; al soltarlo la aplicación envía STOP. El operador observa la respuesta física, porque el movimiento puede continuar un poco después de retirar la señal por la inercia del chasis.

Si el carro no responde a la parada desde la aplicación o sale del área prevista, cortar la alimentación de motores mediante el interruptor del portapilas. Después se investiga la causa con el vehículo detenido. La parada de software, el watchdog y el láser son capas útiles, pero no sustituyen el control físico ni una zona despejada.

11.6 Preparar y ejecutar una ruta

En Rutas se colocan los puntos sobre la cuadrícula y se comprueba que el inicio y la dirección correspondan al montaje físico. Cada celda equivale a 20 cm nominales. Simular permite comprobar si los puntos forman la trayectoria deseada en el modelo; no valida la tracción ni la detección real de un obstáculo.

Antes de ejecutar se revisan la calibración guardada, la confirmación de AUTO, el estado de sensores y la fuente de imagen. Si se va a utilizar la detección por color, la referencia del piso debe corresponder a la superficie del lugar y el área de referencia debe estar despejada. La aplicación muestra el progreso, pero la persona sigue observando el vehículo y el entorno durante toda la ruta.

Si aparece un obstáculo, el navegador puede detenerse y planear una maniobra. No debe colocarse una persona en la trayectoria para probar una detección sin un procedimiento seguro. Si el comportamiento no coincide con lo previsto, se detiene la prueba y se registra lo sucedido, incluidas las intervenciones del operador. La corrección de reincorporación J15 permanece sin validación física.

11.7 Cierre de una sesión

Al terminar, detener la ruta y apagar el portapilas de motores. Después se desconectan las alimentaciones USB de las placas si el vehículo no se va a usar. Se registran cambios observados en la conexión, sensores, batería y comportamiento, junto con el número de corridas realizadas.

La regla de no superar 20 corridas o 90 minutos con el mismo juego de pilas busca mantener comparables las condiciones de ensayo. No equivale a una especificación medida de autonomía. Si se cambian pilas, piso o carga, se revisa la calibración antes de comparar nuevas mediciones con las anteriores.

Dónde está en el código: `ControlLogica.kt`, `ModoSemiautonomo.kt`, `carro.c`, `sensores.c`.

12. Pruebas y resultados

Los resultados se interpretan según el entorno en el que se obtuvieron. Una prueba en computadora comprueba una parte de la lógica; una prueba física incluye además el vehículo, el piso y las condiciones de operación.

12.1 Pruebas físicas registradas

El registro del ajuste MANUAL describe una recta de 150 cm y un desvío de 3 cm con los valores 252 y 255. En pruebas posteriores se observaron diferencias de rumbo y comportamiento durante esquives. En una de cuatro corridas históricas con caja se observó que el carro la evitó pero terminó aproximadamente 40 cm a la derecha; otra probablemente tocó la caja. Estos resultados corresponden a la versión y condiciones de esas pruebas, no a la corrección posterior.

En el registro del ciclo del firmware se observaron máximos de 8.431 ms en reposo con la app conectada y de 6.775 ms en una prueba en piso, ambos con excesos del umbral de 5 ms. La muestra es reducida y la causa no quedó demostrada.

12.2 Software y simulación

El reporte J15 registra 1598 casos del núcleo y 447 de firmware sin fallos en las ejecuciones documentadas. También registra una prueba de estrés de 100000 comandos sin pérdida de STOP. Esos resultados son registros históricos de software; no son mediciones físicas del vehículo.

El escenario de esquive **SIMULADO** usó 400 semillas por versión. Tras la corrección, el error lateral medio fue 1.9 cm y el percentil 95 fue 4.6 cm; antes fueron 13.2 y 21.9 cm. El modelo usa supuestos y no reproduce todos los efectos del piso ni toda la percepción real.

12.3 Pendientes físicos

La corrección J15 no tuvo corridas físicas con movimiento. El criterio de cinco de seis corridas sin contacto y con desvío final menor de 25 cm está **NO MEDIDO**. También permanecen sin medir el balance AUTO, el consumo eléctrico y la eficacia del circuito asociado a GPIO 2.

12.4 Qué demuestra cada clase de ensayo

Una prueba unitaria ejecuta una parte del programa con entradas conocidas y compara la salida con una expectativa. Puede comprobar, por ejemplo, cómo se transforma una ruta o cómo se responde a un mensaje. Cuando el hardware se reemplaza por funciones falsas, la prueba puede reproducir estados y tiempos controlados, pero no incluye el comportamiento eléctrico de la placa.

Una prueba con servidor falso sustituye al carro en el canal de comunicación. Es útil para verificar que la aplicación envíe mensajes, interprete respuestas y maneje una desconexión. No demuestra que una red real tenga la misma latencia, cobertura o pérdida de paquetes, ni que los motores ejecuten el movimiento esperado.

Una simulación de ruta agrega un modelo de vehículo y de obstáculos. Permite repetir una misma geometría con variaciones de parámetros y observar estados, comandos y posiciones virtuales. Su alcance depende de las simplificaciones del modelo: si no representa reflejos, deslizamiento o una falla de alimentación, no puede utilizarse para descartar esos problemas en el carro real.

Una prueba física observa el sistema completo, pero también depende de las condiciones del ensayo. Una sola corrida no caracteriza todas las superficies, baterías y

obstáculos. Para que otra persona pueda interpretar un resultado se necesita registrar fecha, versiones, modo, estado de sensores, condiciones de pista, mediciones y cualquier interrupción.

12.5 Lectura de las cifras de simulación

El escenario B6 del reporte posterior a la corrección usa una ruta recta con una caja modelada y 400 semillas por versión. La semilla permite repetir las variaciones introducidas por el simulador. Se informaron, entre otras métricas, error lateral medio, percentil 95, contactos y rutas terminadas.

El error lateral medio se calcula promediando los errores laterales absolutos de las corridas. El percentil 95 es un valor que permite describir la parte alta de esa distribución: en términos generales, 95 % de las observaciones quedan en o por debajo del percentil calculado. Ninguna de estas dos cifras mide por sí sola el error de avance longitudinal.

Antes de la modificación, la media fue 13.2 cm y el percentil 95 fue 21.9 cm; después, 1.9 y 4.6 cm. Ambos conjuntos registraron 400 rutas terminadas y cero contactos en ese modelo. Con error inferior a 20 cm, terminaron 344 de 400 antes y 400 de 400 después. Diferenciar estos resultados evita confundir una llegada al destino con el cumplimiento de un umbral de precisión.

El criterio físico pendiente usa una definición distinta: al menos cinco de seis corridas sin contacto y con desvío final menor de 25 cm. No puede concluirse que ese criterio se cumple a partir del resultado de 400 simulaciones con un umbral de 20 cm. Se necesitan las corridas físicas y sus registros para responderlo.

12.6 Condiciones de las pruebas físicas anteriores

El reporte histórico de cuatro corridas con una caja sitúa el obstáculo aproximadamente a un metro. Describe una posible colisión en una corrida y una llegada aproximadamente 40 cm a la derecha en otra. En otras corridas se produjeron esquives adicionales. El reporte no conserva todos los datos necesarios para reconstruir cada fase; por eso no atribuye de forma demostrada el resultado a una única causa.

La comparación con versiones posteriores requiere conservar esa limitación. El defecto de pose detectado en la revisión es una explicación técnica plausible de parte de la discrepancia, pero no está demostrado que explique por sí solo toda la desviación observada. Diferencias de giro, calibración, deslizamiento y detección también pueden intervenir.

12.7 Cómo registrar una repetición futura

Antes de una prueba se identifica la versión de la aplicación y, cuando sea posible, la versión leída de la placa. Se registra si la autoridad del láser está habilitada, la fuente de imagen, la calibración, el estado de las baterías y la geometría de la pista. El operador debe conocer el procedimiento de parada y mantener libre el área de maniobra.

Para cada corrida se anotan el inicio, la ruta, la ubicación del obstáculo, si hubo contacto y el desvío final medido con una referencia externa. También conviene conservar los

registros de pose y de giro con sus tiempos. Si la corrida se aborta o se toca el carro, se anota esa intervención: excluirla silenciosamente puede distorsionar las conclusiones.

Al resumir, se informa el número total de intentos, cuántos se completaron y qué criterio se usó para considerarlos exitosos. Las unidades se mantienen junto a cada resultado. Si faltan versiones, condiciones o datos de una corrida, el informe debe señalarlo en lugar de completar los huecos con suposiciones.

12.8 Preparar una ficha de ensayo

Antes de cada prueba, asigne un identificador y anote fecha, persona responsable, revisión del software y dispositivo utilizado. Indique también el modo de conducción, el estado de carga conocido, la superficie, el trazado, el tipo de obstáculo y la iluminación cuando intervenga la cámara. Si algún dato no se conoce, regístrelo como desconocido en lugar de estimarlo sin evidencia.

Describa qué se quiere medir y cómo se hará. Por ejemplo, para desvío lateral es necesario definir una línea de referencia, un punto inicial, un punto final y la regla para medir perpendicularmente el desplazamiento. También hay que indicar la resolución del instrumento utilizado y si el carro terminó la maniobra o tuvo que detenerse.

Una ficha debe separar el resultado observado de la interpretación. «Se detuvo antes de llegar a la marca» describe una observación. «El sensor frenó correctamente» es una hipótesis que requiere comprobar la lectura, la configuración y el evento de parada. Si hay un registro de mensajes, consérvelo junto a la ficha, retirando datos sensibles.

12.9 Ensayo propuesto de movimiento recto

Seleccione una zona despejada y trace una referencia recta sobre el piso. Marque el punto de partida y el objetivo, coloque el carro con una orientación inicial comparable y defina por adelantado quién supervisa la parada. Registre el modo, la calibración y las condiciones de alimentación. Realice la prueba a una distancia apropiada para el espacio disponible.

Al finalizar, mida la distancia longitudinal y el desvío lateral respecto de la referencia. Anote si hubo intervención humana, contacto, patinamiento visible o interrupción. Si se repite, vuelva a colocar el vehículo en la misma posición y orientación. No mezcle resultados de superficies o calibraciones diferentes sin indicarlo.

Para interpretar los datos, compare los resultados individuales además de cualquier promedio. Un promedio pequeño puede ocultar corridas con desvíos grandes en sentidos opuestos. Este ensayo caracteriza solo la configuración y condiciones efectivamente probadas; no sustituye una evaluación de esquive.

12.10 Ensayo propuesto de giro

Prepare una marca de orientación inicial y una referencia angular en una zona libre. Con el carro inmóvil, complete la preparación del giroscopio según la operación del sistema. Solicite un giro admitido y observe tanto la respuesta comunicada como el ángulo final medido externamente, si se dispone de un método de medición.

Registre el sentido del giro, el ángulo solicitado, la respuesta y cualquier intervención. Una respuesta de finalización indica el resultado de la lógica de control, mientras que la

comparación con una referencia externa permite estimar el error físico. Si aparece un tiempo agotado o una falla de sensor, no se contabiliza como giro preciso completado.

Las repeticiones ayudan a identificar dispersión, pero no prueban todos los ángulos y condiciones. Si se cambian las ruedas, el piso o la alimentación, esas condiciones deben anotarse en una serie distinta.

12.11 Ensayo propuesto de video y control

Con el vehículo inmovilizado de forma segura, compruebe por separado la conexión de control y la recepción de video. Registre el estado inicial y observe qué indica la aplicación cuando se interrumpe cada enlace por separado. Para comprobar si una imagen se actualiza, cambie un objeto visible frente a la cámara y confirme que el cambio aparece.

Esta comprobación permite detectar que un canal responde o deja de responder en el escenario ensayado. No mide la latencia completa salvo que se utilice un procedimiento con una referencia temporal y puntos de inicio y fin definidos. Tampoco demuestra la cobertura inalámbrica en otras ubicaciones.

12.12 Ensayo propuesto de parada frontal

El ensayo de protección frontal requiere una pista controlada, supervisión directa y una forma de detener el carro. Antes de comenzar, confirme el modo y que el uso del láser esté habilitado. Coloque un obstáculo apropiado y mida su distancia con una referencia independiente, sin colocar personas en la trayectoria.

Registre la lectura del sensor cuando sea posible, el mensaje de parada y la posición final. Si la distancia reportada no es válida o el sistema se comporta de forma inesperada, detenga la prueba. Una respuesta positiva en una geometría concreta no demuestra detección de cualquier material, ángulo o altura.

12.13 Análisis de una serie de resultados

Conserve todas las corridas, incluidas las fallidas y las interrumpidas, e indique por qué se excluye alguna de un cálculo. Para una distancia medida, informe la unidad, el número de resultados y una medida de dispersión o los valores individuales. Si se calcula una proporción, indique claramente el numerador y el denominador.

Por ejemplo, cinco éxitos entre seis intentos se expresan como 5/6, no como una garantía de éxito futuro. Un criterio puede exigir simultáneamente ausencia de contacto y desvío menor que un límite: si falta una de las mediciones, no corresponde contar la corrida como cumplimiento demostrado.

12.14 Comparación entre revisiones

Para comparar una corrección con una versión anterior, mantenga el procedimiento y las condiciones lo más constantes posible. Si cambia la posición del obstáculo, la superficie o el estado de carga, anótelo como diferencia. Cuando sea viable, alterne el orden de las versiones para reducir el efecto de cambios durante la sesión, y registre cualquier excepción.

La comparación de J15 disponible en el repositorio corresponde a simulación con las semillas y el modelo descritos en el reporte. Una comparación física futura deberá identificar sus propias condiciones y no atribuir automáticamente los valores simulados al vehículo real.

Dónde está en el código: `pruebas_reincorporacion_j15.c`, `simulador_navegador.c`, `config.h`.

13. Límites, riesgos y trabajo futuro

La navegación depende de una estimación de movimiento y de sensores con cobertura limitada. Los resultados deben interpretarse dentro de las condiciones en que se registraron y con supervisión física del vehículo.

La distancia recorrida se calcula por tiempo. El estado de las pilas, la superficie, la carga y el deslizamiento pueden modificar el desplazamiento real. El giroscopio observa orientación, pero no identifica directamente un deslizamiento lateral. La cámara puede verse afectada por iluminación y sombras; el láser mira en una dirección limitada.

El circuito de mantenimiento de la powerbank en GPIO 2 está configurado en software, pero su carga y eficacia están **NO MEDIDAS**. Tampoco hay medición suficiente del consumo total, la autonomía o las corrientes de bloqueo. El balance AUTO requiere una medición específica; no debe tratarse como calibración física ya validada.

El trabajo futuro prioritario es medir la corrección J15 en un entorno controlado, comparar la posición real con la pose estimada y registrar contactos y desvíos. También deben medirse las corrientes del sistema, comprobarse el circuito de GPIO 2 y caracterizarse el balance AUTO. La repetición debe conservar las condiciones y versiones para que los resultados sean comparables.

13.1 Alcance del entorno de operación

El sistema debe usarse en un espacio que el operador pueda observar y despejar. La cámara y el sensor frontal no sustituyen una inspección del recorrido. Objetos bajos, transparentes, estrechos o situados fuera de la zona observada pueden no generar una señal útil en las condiciones reales; su detección no debe darse por garantizada sin una prueba específica.

La iluminación puede alterar la imagen y, con ella, la clasificación de regiones. Un cambio en el color o la textura del piso también puede modificar la interpretación del entorno. La respuesta práctica es verificar el comportamiento en el entorno concreto y detener la operación cuando la información visual deja de ser confiable.

13.2 Precisión y repetibilidad

La calibración de tiempo por distancia relaciona una orden con un desplazamiento observado bajo determinadas condiciones. Si cambian las pilas, la carga o la superficie, la misma duración puede producir otra distancia. Por ello un valor de calibración no debe interpretarse como una garantía universal de exactitud.

El giro estimado por giroscopio también depende de las condiciones de medición y del sesgo del sensor. Una calibración hecha mientras el vehículo se mueve puede

contaminar la referencia. Incluso con un rumbo razonable, el carro puede desplazarse lateralmente por deslizamiento o por diferencias entre ruedas.

13.3 Evidencia pendiente y prioridad

Para J15, el siguiente paso útil es repetir físicamente la maniobra con obstáculos, ruta y condiciones registradas, midiendo el desvío final y anotando contactos. La comparación debe identificar versión y configuración, porque una ejecución de una revisión anterior no valida una corrección posterior. Hasta disponer de esas corridas, el resultado físico continúa **NO MEDIDO**.

Para el balance AUTO se necesita una prueba física específica; el ajuste registrado para MANUAL no demuestra el comportamiento de AUTO. Para el consumo conviene distinguir alimentación de placas, motores en movimiento y situaciones de carga, siempre con instrumentación adecuada. Sin datos de corriente y capacidad utilizable no se calcula una autonomía confiable.

La función del GPIO 2 debe revisarse en el circuito real y medirse antes de afirmar que mantiene encendida una batería portátil. Una configuración en el código documenta la intención de generar una señal; no demuestra por sí sola la corriente, el efecto en la batería ni la seguridad del montaje concreto.

13.4 Cambios futuros y regresiones

Cuando se modifique una regla de navegación, conviene repetir las pruebas de software relacionadas y comparar las salidas con las de la revisión anterior. Si el cambio altera el movimiento físico, las pruebas en computadora son una primera barrera, no la última: después se requiere una prueba controlada en el carro con una persona preparada para detenerlo.

Una mejora en una métrica no confirma automáticamente mejoras en todas las demás. Por ejemplo, reducir el error lateral simulado no mide consumo, autonomía, cobertura de cámara ni confiabilidad del enlace inalámbrico. Cada afirmación debe acompañarse de su propia evidencia.

Dónde está en el código: `navegador.c`, `config.h`, `tof.c`, `obstaculos.c`.

14. Proceso de desarrollo

El proyecto evolucionó desde un control de carro basado en ESP32 hacia una arquitectura distribuida con pruebas de software y sensores. Esta sección resume las decisiones que ayudan a comprender el diseño actual.

La lógica de control se separó de las funciones de hardware para poder ejecutar pruebas con implementaciones simuladas. El núcleo de navegación se escribió en C y se integró en Android, lo que permite reutilizar la lógica en computadora y teléfono sin mantener dos algoritmos independientes.

Después se incorporaron el giroscopio y el sensor frontal de distancia. El giroscopio permite cerrar giros por ángulo; el láser agrega una condición de parada en avance. La cámara Freenove quedó en otra placa porque la captura y la compresión tienen un costo de procesamiento distinto del control de tracción.

Las pruebas físicas mostraron variaciones de movimiento y de orientación. La calibración manual redujo el desvío de una recta en las condiciones registradas. En el análisis de reincorporación posterior se identificó que la estimación proyectaba los avances con un rumbo fijo. Se cambió la integración para aprovechar el rumbo observado durante el avance y se probó la corrección en simulación. Su desempeño físico continúa pendiente.

El diseño no dispone de encoders ni de una referencia externa de posición. La elección de avance por tiempo simplifica el montaje y permite trabajar con los componentes disponibles, pero exige calibración y deja incertidumbre acumulada. Las simulaciones ayudan a explorar la lógica sin mover el carro; no reemplazan la validación de las hipótesis sobre tracción y percepción.

14.1 Separar el control del hardware

En el prototipo inicial, la lógica de movimiento, el servidor y el acceso a pines estaban concentrados en el programa de la placa. La reorganización introdujo una interfaz de hardware para que la lógica de control pudiera invocarse con funciones reales en la ESP32 o con funciones falsas en computadora. Esto permitió ensayar estados y respuestas sin que cada prueba necesitara un vehículo energizado.

El mismo criterio se aplicó al núcleo de navegación. Las operaciones de ruta y calibración se conservaron en C; la aplicación Android las invoca mediante una interfaz nativa. Así se puede ensayar la lógica con un simulador y, a la vez, usarla en la interfaz del teléfono. Las pruebas de cada capa comprueban propiedades distintas: una suite en computadora no incluye automáticamente el comportamiento de WiFi, ruedas y piso.

14.2 Incorporar sensores y calibrar

La navegación original dependía del tiempo para avanzar y girar. El giroscopio incorporó una observación de velocidad angular para controlar los giros por ángulo; el sensor de distancia frontal añadió una condición de parada en avance. La separación entre ambos sensores importa: el giroscopio observa orientación y el láser una distancia en su dirección de montaje.

El ajuste de balance MANUAL responde a una asimetría observada en el carro. En una recta de 150 cm el comentario de configuración registra 14.5 cm de desvío con ambos lados a 255 y 3 cm con el ajuste de 252 a la izquierda. El valor AUTO se obtuvo al escalar la relación y permanece sin medición física. La separación entre un parámetro calculado y uno medido evita atribuir a ambos la misma evidencia.

14.3 Cámara y procesamiento local

El GC0308 entrega cuadros que requieren compresión JPEG por software. El proyecto probó combinaciones de formato, reloj y resolución para evaluar el costo de captura y compresión. La configuración actual usa HQVGA; el barrido local permitió comparar su rendimiento con QVGA bajo condiciones concretas. La decisión intercambia detalle espacial por una mayor frecuencia y menor edad de cuadro en ese ensayo.

Mantener la placa de cámara separada de la DevKit reduce la interacción directa entre el trabajo de video y el control de motores. El teléfono recibe el video y ejecuta el procesamiento necesario para la aplicación. Este reparto también introduce dependencia

de dos enlaces: la disponibilidad del video y la del control deben observarse por separado.

14.4 Diagnóstico de reincorporación

En las pruebas históricas con una caja se observaron desviaciones y esquives adicionales. La revisión posterior encontró que la estimación proyectaba un avance completo usando un rumbo fijo, aunque el giroscopio transmitía información durante el recorrido. El cambio integra incrementos con el rumbo observado cuando la lectura es válida; no convierte la distancia por tiempo en una distancia medida.

Para comparar el cambio se ejecutaron escenarios simulados con una posición virtual independiente de la estimación. La mejora del error lateral en ese modelo respalda la modificación de la lógica, pero los propios registros señalan que el escenario no reproduce exactamente las desviaciones históricas ni toda la percepción real. La pregunta sobre la mejora en el carro queda abierta hasta realizar el ensayo físico.

14.5 Alternativas y compromisos

El diseño utiliza la red local y el teléfono para la interacción y el cálculo, sin requerir un servidor externo para el recorrido. No hay encoders ni localización externa; por ello la distancia depende de la calibración y no puede corregirse todo deslizamiento. El láser agrega una parada frontal independiente de la clasificación de imagen, pero no cubre todo el entorno y su autoridad es configurable.

El arranque a potencia plena durante un intervalo corto responde al comportamiento observado cuando los motores zumbaban sin arrancar a valores menores. Una rampa progresiva puede ser deseable en otros montajes, pero el cambio de estrategia debe evaluarse con el vehículo y sus cargas reales. Estas decisiones son compromisos del prototipo, no garantías universales de desempeño.

La Figura 11 presenta: Etapas del desarrollo del sistema.

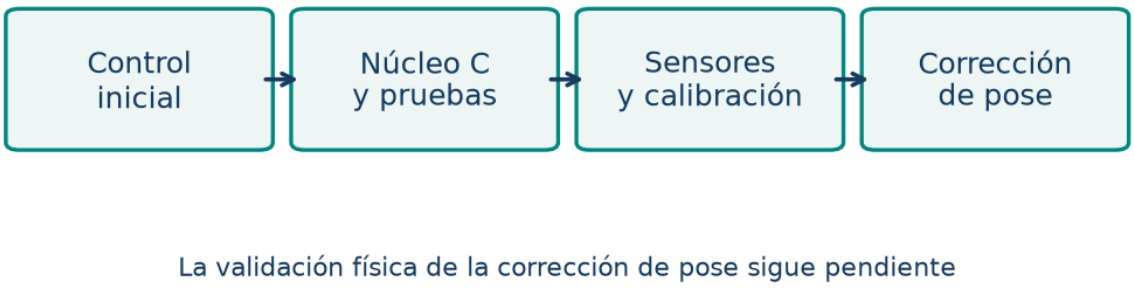


Figura 11. Etapas del desarrollo del sistema.

Dónde está en el código: `carro.c`, `trayectoria.c`, `navegador.c`, `simulador_navegador.c`.

15. Solución de problemas

El diagnóstico comienza por la alimentación y las conexiones, con los motores apagados. Una indicación en la aplicación ayuda a orientar la búsqueda, pero no demuestra por sí sola el estado físico de todos los componentes.

15.1 Red y video

Si la red del carro no aparece, comprobar que la DevKit recibe energía y que su cable USB funciona. Si la app no conecta, revisar que el teléfono esté en CarroFSI y que la dirección del carro sea 192.168.4.1. Un segundo cliente de control puede desplazar la conexión anterior.

Si hay control pero no video, comprobar la batería propia de la cámara, su conexión a la red y la dirección 192.168.4.2. La cámara atiende un cliente de video a la vez. Si el teléfono está fuera de la red, la pantalla puede indicar reconexión hasta que vuelva a conectarse.

15.2 Movimiento y sensores

Si la aplicación indica una orden pero los motores no responden, detener la operación y revisar el interruptor del portapilas, las pilas y la tierra común entre DevKit y driver. Si el carro se desvía, comprobar ruedas y cableado y revisar el balance; el ajuste AUTO no cuenta con validación física registrada.

Si el rumbo cambia mientras el carro está inmóvil, verificar que permanezca quieto durante la calibración del giroscopio. Si el láser no responde, revisar alimentación y conexiones I2C. No continuar una ruta mientras la información necesaria para supervisarla sea poco confiable.

15.3 Método de diagnóstico

Describa primero el síntoma: qué se esperaba, qué ocurrió y en qué modo. Registre si hay imagen, si existe conexión de control y si el problema afecta a ambos lados del carro. Anote también cambios recientes, como pilas sustituidas, cables movidos o una nueva versión de software. Una descripción concreta evita confundir síntomas de componentes distintos.

Compruebe una hipótesis por vez. Si se cambia simultáneamente la alimentación, la red y el programa, puede desaparecer el problema sin que se conozca su causa. Después de cada cambio, repita una prueba breve y anote el resultado. Si hay movimiento inesperado, detenga el carro y retire la energía antes de inspeccionar conexiones.

15.4 No aparece la red del carro

Compruebe la fuente USB y el cable de la DevKit. Un indicador luminoso puede ayudar a ver que llega energía, pero no confirma que el programa haya iniciado correctamente ni que la red esté disponible. Si se dispone de acceso al monitor serie, revise los mensajes de arranque sin divulgar credenciales.

Si la red aparece intermitentemente, registre en qué momento se interrumpe y si coincide con movimientos o cambios de alimentación. No atribuya de inmediato el fallo a una causa concreta: una batería, un cable, el entorno inalámbrico o el propio programa requieren comprobaciones diferentes.

15.5 La red existe, pero no hay control

Verifique que el teléfono esté realmente conectado a la red local del carro y que la aplicación use la dirección configurada. Observe si el estado cambia durante el intento de conexión. Si se conecta otro cliente de control, el comportamiento del servidor puede desplazar una sesión anterior; evite interpretar ese cambio como un fallo mecánico.

Si se obtiene respuesta a una consulta pero no se produce movimiento, la red proporciona una pista de que hay comunicación, no una garantía de que el driver y sus motores reciban energía. Pase entonces al diagnóstico de la cadena de movimiento.

15.6 Hay control, pero falta imagen

Revise la batería portátil de la cámara, su cable y su arranque. Compruebe que la cámara se haya unido a la red del carro y que la aplicación intente acceder a su dirección. Una sola conexión de video admitida puede hacer que otra sesión compita por el servicio. Cierre las sesiones de prueba que no se utilicen antes de repetir.

Una imagen antigua no equivale a una imagen actual. Para valorar el flujo, observe si cambia al mover un objeto en un entorno seguro y revise las indicaciones de la aplicación. Si el modo de navegación depende de cámara, no continúe con una imagen ausente o evidentemente congelada.

15.7 El carro no se mueve

Detenga la orden, apague la alimentación y revise la sujeción de ruedas y cables. Compruebe el portapilas, su interruptor, la conexión al driver y la referencia común con la DevKit. No conecte ni desconecte el cableado de motores con el sistema energizado. Una batería USB con carga no demuestra que las pilas de motores tengan carga.

Si solo responde un lado, compare las conexiones y la mecánica de ambos lados con el esquema del proyecto. Una diferencia puede encontrarse en el motor, la transmisión, el cableado o el canal del driver. Evite invertir conexiones al azar: documente la posición original y compruebe cada cambio de forma controlada.

15.8 El carro se desvía

Revise si alguna rueda roza, si hay cables frenando el movimiento o si el piso produce adherencia desigual. Después compare el modo y la calibración usados con los del registro disponible. El ajuste MANUAL medido no equivale a una validación física de AUTO.

Para comparar dos ajustes, utilice la misma distancia de prueba, superficie y condiciones de carga; marque el punto inicial y mida el desvío al final. Repita varias veces y conserve los resultados individuales. Una sola trayectoria puede estar influida por una irregularidad puntual del piso.

15.9 La orientación cambia con el carro quieto

Asegúrese de que el carro permanezca inmóvil durante la calibración del giroscopio y revise el estado que informa la aplicación. Si el valor cambia, anote durante cuánto tiempo y si vuelve a estabilizarse. Una lectura de orientación no mide directamente la posición lateral; no debe emplearse por sí sola para concluir cuánto se desplazó el carro.

15.10 El sensor frontal no responde o parece inconsistente

Con la alimentación desconectada, inspeccione el cableado y las conexiones del bus I2C según el esquema. Vuelva a energizar y consulte el estado del sensor. Una respuesta ausente, una lectura fuera de rango y una distancia válida son situaciones distintas que deben registrarse por separado.

Si la lectura parece válida pero el carro no se detiene como se espera, compruebe el modo y si el uso del láser está habilitado. La disponibilidad del sensor y la autoridad para detener el avance son conceptos diferentes. Mantenga un espacio seguro y no pruebe la protección acercando personas al carro.

15.11 La ruta no termina donde se esperaba

Guarde el trazado, la versión, la calibración y la posición de partida. Compare después la trayectoria observada con la estimada, incluyendo dónde ocurrió una evasión. El error puede acumularse en tramos rectos, giros o maniobras de esquive; el punto final por sí solo no identifica cuál fue la causa.

Si se está evaluando J15, no se debe sustituir una medición física por el resultado simulado. Registre contactos y desvío final con un procedimiento repetible y compare las corridas con el criterio indicado en el capítulo de pruebas.

Dónde está en el código: `firmware_carro.ino`, `sensores.c`, `Enlace.kt`, `SesionVideo.kt`.

16. Anexos

Los anexos reúnen términos y datos de consulta para usar junto con los capítulos anteriores. Los valores configurados no sustituyen las mediciones físicas pendientes.

16.1 Glosario

ESP32: familia de microcontroladores con conectividad inalámbrica. **I2C:** bus compartido para comunicar circuitos mediante líneas de datos y reloj. **PWM:** señal cuyo ciclo de encendido regula la potencia aplicada. **Pose:** posición y orientación estimadas del vehículo. **WebSocket:** canal de comunicación bidireccional persistente. **Watchdog:** mecanismo que vigila un plazo y ejecuta una acción si no se cumple la condición esperada. **ToF:** medición de distancia basada en el tiempo de vuelo de la luz.

16.2 Comandos principales

La Tabla 1 resume las órdenes principales. La activación de la autoridad del láser se controla con `T0F_US0`.

Comando	Función
FORWARD / REVERSE	Avanzar / retroceder
LEFT / RIGHT	Pivote izquierdo / derecho
STOP / EBRAKE	Detener motores
PING	Sondeo con respuesta PONG; no renueva el temporizador
MODO_AUTO / MODO_MANUAL	Seleccionar perfil
GIRAR:<grados>	Solicitar giro por ángulo
CAPS? / IMU? / TOF?	Consultar estado de sensores
IMU_CAL / IMU_CERO	Calibrar sesgo / fijar referencia angular

Tabla 1. Comandos de consulta y control.

16.3 Pines de la DevKit

La Tabla 2 reúne la asignación de los pines empleados por el controlador.

GPIO	Función
27 / 26	Entradas del driver, lado izquierdo
25 / 33	Entradas del driver, lado derecho
21 / 22	I2C: SDA / SCL
2	Pulso de mantenimiento configurado; efecto físico no medido

Tabla 2. Asignación de pines.

16.4 Fuentes por tema

Arquitectura y hardware: `conexiones.md`, `config.h`. Firmware y protocolo: `carro.c`, `sensores.c`, `protocolo.md`. Cámara: `camara_prueba.ino`, `camara_prueba.md`. Navegación: `navegador.c`, `calibracion.c`, `reporte_J15.md`. Percepción: `obstaculos.c`, `fusion.c`, `corredor.c`. Aplicación: `MainActivity.kt`, `ModoSemiautonomo.kt`. Resultados físicos: `reporte_J13_fisico.md`, `reporte_J14.md`.

16.5 Pendientes

Validar físicamente el esquive y la reincorporación J15; medir el balance AUTO, el consumo eléctrico y el comportamiento de GPIO 2; comprobar la versión de firmware en la placa cuando se disponga del equipo.

16.6 Glosario ampliado: electrónica y energía

Alimentación: suministro de energía a un circuito. En este proyecto deben distinguirse la alimentación USB de cada placa y la de los motores mediante pilas y driver. Que una fuente funcione no garantiza el funcionamiento de las demás.

Balance: ajuste relativo de potencia entre los lados del carro para reducir una desviación. Su valor depende del perfil y sus resultados físicos deben evaluarse en las

condiciones de prueba. Un valor guardado no equivale, por sí mismo, a un ajuste verificado en todos los pisos.

Bus I2C: medio compartido en el que dispositivos intercambian datos utilizando las líneas SDA y SCL. Los dispositivos deben ser compatibles con las condiciones eléctricas del bus y tener conexiones correctas. El hecho de compartir líneas facilita el cableado, pero una falla común puede afectar varias lecturas.

Ciclo de trabajo PWM: proporción del periodo durante la cual una señal está activa. El firmware utiliza valores para controlar la potencia aplicada por el driver. El efecto mecánico no depende solo de ese número: también intervienen la batería, el motor, la carga y la fricción.

Driver o puente H: etapa de potencia que permite gobernar los motores a partir de señales de control. Separa la función de decisión del microcontrolador de la conmutación de energía para los motores. Debe estar correctamente alimentado y conectado a la referencia común indicada para el montaje.

GPIO: pin de entrada o salida de propósito general del microcontrolador. La función concreta de un pin depende de la configuración del firmware y de su conexión física. La presencia de una definición en el código no confirma que el circuito asociado esté montado o medido.

Tierra común: referencia eléctrica compartida entre circuitos que intercambian señales. En el carro se indica para la DevKit, el driver y el negativo del portapilas de motores. No debe confundirse con unir indiscriminadamente todas las fuentes sin revisar el esquema.

16.7 Glosario ampliado: comunicaciones

Cliente y servidor: un servidor ofrece un servicio y un cliente inicia una conexión para utilizarlo. Una misma aplicación puede ser cliente de servicios distintos, como control y video. La disponibilidad de un servicio no demuestra la disponibilidad de los demás.

Dirección IP: identificador de un dispositivo en una red concreta. Las direcciones descritas para la DevKit y la cámara pertenecen a la configuración del proyecto. Una dirección correcta ayuda a encontrar el servicio, pero no verifica la versión que ejecuta el dispositivo.

Latencia: tiempo transcurrido entre un evento y su observación o respuesta. Para interpretar una cifra hay que conocer sus puntos de inicio y fin. El tiempo de captura de la cámara no representa necesariamente toda la demora entre la escena y la pantalla del teléfono.

Telemetría: información que un dispositivo comunica sobre su estado o sus sensores. En el perfil correspondiente, los mensajes SENS reúnen valores de orientación, distancia y banderas. Una lectura se interpreta junto con las banderas y el estado de la conexión.

WebSocket: canal de comunicación bidireccional sobre una conexión persistente. Puede transportar órdenes y respuestas sin abrir una conexión nueva por cada mensaje. Una sesión activa necesita vigilancia de desconexiones y de datos atrasados.

Watchdog: mecanismo de vigilancia de un plazo. Si se cumple su condición de vencimiento, el firmware aplica la acción programada. Su comportamiento debe describirse con el código concreto: en este proyecto no todos los mensajes tratan el temporizador de la misma manera.

16.8 Glosario ampliado: movimiento y percepción

Calibración: procedimiento para relacionar una señal o parámetro con una referencia. La calibración del tiempo de desplazamiento y la del sesgo del giroscopio resuelven problemas distintos. En ambos casos, las condiciones de la medición influyen en el resultado.

Giroscopio: sensor que mide velocidad angular. Integrando esa información se estima el cambio de orientación. El sesgo y otros errores pueden acumularse; el sensor no mide directamente la distancia recorrida ni la posición lateral.

Modelo de piso: representación usada por el procesamiento de imagen para decidir qué regiones se parecen al suelo aprendido. La interpretación puede verse afectada por cambios de iluminación, textura y composición de la escena. El modelo no identifica de forma infalible todos los objetos.

Pose: combinación de posición y orientación empleada para representar el estado del vehículo. En este proyecto la pose es una estimación calculada por software. Debe compararse con una referencia física si se quiere cuantificar su error real.

Reincorporación: maniobra destinada a regresar hacia la ruta después de apartarse de ella. En J15 existen resultados de simulación de la corrección de pose y reincorporación; la validación física de esa revisión permanece pendiente.

Sensor ToF: sensor que estima distancia a partir del tiempo de vuelo de la luz. Su lectura depende del alcance y de las condiciones del objeto observado. Su orientación frontal no proporciona una cobertura completa alrededor del carro.

16.9 Glosario ampliado: pruebas

Caso de prueba: conjunto de condiciones, acciones y resultado esperado usado para comprobar un comportamiento. Debe poder repetirse y distinguir claramente el resultado observado del esperado.

Percentil 95: valor por debajo o igual al cual se encuentra aproximadamente el 95 % de una muestra ordenada. Resume la parte alta de una distribución, pero no es el máximo y no explica por sí solo cómo se recogieron los datos.

Simulación: ejecución con un modelo que representa parte del sistema o su entorno. Permite repetir escenarios y explorar variaciones, pero sus conclusiones están limitadas por los fenómenos que el modelo incluye.

Trazabilidad: capacidad de relacionar un resultado con su versión, configuración, procedimiento y condiciones. Facilita repetir la prueba y evita atribuir a una revisión un comportamiento observado en otra.

Validación física: comprobación en el equipo real bajo condiciones registradas. Su alcance depende del número y variedad de pruebas; una ejecución satisfactoria no demuestra funcionamiento en todas las condiciones posibles.

17. Referencia técnica de archivos

Esta referencia complementa la explicación por subsistemas. Identifica archivos de implementación propios, sus responsabilidades y sus relaciones principales. Un archivo de cabecera declara una interfaz; el archivo C correspondiente implementa esa interfaz. El nombre de un archivo describe su función en esta revisión, no garantiza que todas sus funciones estén activas en todos los modos.

Los archivos de recursos gráficos, bibliotecas externas y productos compilados se explican por familia al final. No se imprimen claves ni el contenido de archivos locales de secretos. Para conocer parámetros y excepciones, consulte también los capítulos de firmware, navegación y pruebas.

17.1 Firmware: arranque, configuración y seguridad

firmware_carro/firmware_carro.ino. Es el punto de entrada de Arduino de la DevKit. Conecta WiFi, servidor, bucle y operaciones reales de hardware con los módulos C; conviene seguirlo para localizar la inicialización y la ejecución periódica.

firmware_carro/config.h. Concentra pines, tiempos y parámetros del firmware. Un cambio aquí puede afectar conexiones, límites o comportamiento; la definición de un valor no demuestra por sí misma que se haya validado en el carro.

firmware_carro/carro.h. Declara la interfaz de la lógica de control y las operaciones que debe proporcionar el hardware. Permite probar decisiones de movimiento sin depender directamente de Arduino.

firmware_carro/carro.c. Interpreta comandos, gestiona perfiles, cola y vigilancia, y decide las órdenes para motores. Revise aquí el tratamiento de STOP y las excepciones del temporizador antes de modificar el protocolo.

firmware_carro/hw_sensores.h. Declara las operaciones de hardware requeridas por la capa de sensores. Separa las decisiones de los detalles concretos de acceso a la placa.

firmware_carro/pagina.h. Contiene la página web integrada que sirve el firmware. Es una interfaz alternativa y sus controles deben interpretarse junto con las órdenes que acepta el controlador.

firmware_carro/pagina.sha256. Guarda una huella de referencia de la página. Su utilidad es comprobar consistencia con la versión esperada, no verificar el estado físico del carro.

firmware_carro/secretos.ejemplo.h. Es una plantilla de configuración local de red. Los valores reales se mantienen fuera de la documentación pública y no deben copiarse al manual.

17.2 Firmware: sensores y giro

firmware_carro/imu.h. Declara la interfaz, las estructuras y los límites del módulo que estima el ángulo a partir de la velocidad angular del MPU6050 y su sesgo. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

firmware_carro/tof.h. Declara la interfaz, las estructuras y los límites del módulo que consulta y clasifica las lecturas de distancia del VL53L0X. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

firmware_carro/giro.h. Declara la interfaz, las estructuras y los límites del módulo que controla una maniobra de giro usando la orientación estimada y un límite de tiempo. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

firmware_carro/sensores.h. Declara la interfaz, las estructuras y los límites del módulo que coordina el giroscopio, el láser y el controlador de giro con la lógica del carro. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

firmware_carro/imu.c. Implementa el módulo que estima el ángulo a partir de la velocidad angular del MPU6050 y su sesgo. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

firmware_carro/tof.c. Implementa el módulo que consulta y clasifica las lecturas de distancia del VL53L0X. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

firmware_carro/giro.c. Implementa el módulo que controla una maniobra de giro usando la orientación estimada y un límite de tiempo. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

firmware_carro/sensores.c. Implementa el módulo que coordina el giroscopio, el láser y el controlador de giro con la lógica del carro. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

17.3 Cámara y recuperación

camara_prueba/camara_prueba.ino. Configura la placa Freenove, la captura y los servicios de video. También coordina la red y la entrega de cuadros; es el punto de partida para investigar formatos, rendimiento y conexión.

camara_prueba/recuperacion.h. Declara la interfaz de las decisiones de recuperación ante fallos de cámara o conexión, separada del hardware.

camara_prueba/recuperacion.c. Implementa decisiones y tiempos de recuperación para que puedan comprobarse en computadora. El resultado depende del estado y de las señales que recibe desde el programa de cámara.

camara_prueba/secretos.ejemplo.h. Es la plantilla de configuración de la red a la que se conecta la cámara. No contiene una demostración de qué configuración está cargada actualmente en una placa.

camara_prueba/src/JPEGENC/. Agrupa una biblioteca externa para compresión JPEG y su licencia. Sus archivos se conservan como dependencia de terceros; no deben atribuirse como implementación original del equipo.

17.4 Núcleo C: ruta, calibración y navegación

nucleo_c/calibracion.h. Declara la interfaz, las estructuras y los límites del módulo que convierte medidas y parámetros de calibración en tiempos de movimiento. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/calgiro.h. Declara la interfaz, las estructuras y los límites del módulo que organiza la calibración de giros con lecturas del giroscopio. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/trayectoria.h. Declara la interfaz, las estructuras y los límites del módulo que transforma puntos de cuadrícula en pasos de avance y giro. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/ruta_editor.h. Declara la interfaz, las estructuras y los límites del módulo que mantiene la lógica de edición de puntos y operaciones sobre la ruta. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/navegador.h. Declara la interfaz, las estructuras y los límites del módulo que ejecuta la máquina de estados de seguimiento, esquive y reincorporación. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/simulador_navegador.h. Declara la interfaz, las estructuras y los límites del módulo que representa un carro y un entorno virtuales para ensayar el navegador. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/sim_ruta.h. Declara la interfaz, las estructuras y los límites del módulo que ejecuta una ruta con el navegador real y el vehículo simulado para obtener su recorrido. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/calibracion.c. Implementa el módulo que convierte medidas y parámetros de calibración en tiempos de movimiento. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/calgiro.c. Implementa el módulo que organiza la calibración de giros con lecturas del giroscopio. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/trayectoria.c. Implementa el módulo que transforma puntos de cuadrícula en pasos de avance y giro. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/ruta_editor.c. Implementa el módulo que mantiene la lógica de edición de puntos y operaciones sobre la ruta. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/navegador.c. Implementa el módulo que ejecuta la máquina de estados de seguimiento, esquive y reincorporación. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/simulador_navegador.c. Implementa el módulo que representa un carro y un entorno virtuales para ensayar el navegador. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/sim_ruta.c. Implementa el módulo que ejecuta una ruta con el navegador real y el vehículo simulado para obtener su recorrido. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

17.5 Núcleo C: percepción y video

nucleo_c/obstaculos.h. Declara la interfaz, las estructuras y los límites del módulo que analiza color de piso y clasifica zonas de la imagen. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/corredor.h. Declara la interfaz, las estructuras y los límites del módulo que proyecta la huella del vehículo sobre la imagen para evaluar la zona de paso. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/cualquiera.h. Declara la interfaz, las estructuras y los límites del módulo que procesa el modo de detección de cualquier objeto del modelo. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/fusion.h. Declara la interfaz, las estructuras y los límites del módulo que combina las señales de color y del modelo de objetos para una decisión. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/mascara_vision.h. Declara la interfaz, las estructuras y los límites del módulo que gestiona una máscara para excluir artefactos fijos de la imagen. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/movimiento_piso.h. Declara la interfaz, las estructuras y los límites del módulo que compara el movimiento de bloques del piso entre cuadros. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/movimiento_visual.h. Declara la interfaz, las estructuras y los límites del módulo que estima giro visual mediante cuadros consecutivos. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/paquete_celdas.h. Declara la interfaz, las estructuras y los límites del módulo que construye e interpreta el paquete UDP de promedios de color por celda. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/pisos_sinteticos.h. Declara la interfaz, las estructuras y los límites del módulo que genera escenas sintéticas de piso para pruebas en computadora. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/prueba_obstaculo.h. Declara la interfaz, las estructuras y los límites del módulo que calcula métricas de retraso y distancia en pruebas de parada por cámara. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/latencia_video.h. Declara la interfaz, las estructuras y los límites del módulo que compara tiempos de respuesta del control con y sin video. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/video_stats.h. Declara la interfaz, las estructuras y los límites del módulo que reúne estadísticas temporales de llegada de cuadros. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/sesion_compartida.h. Declara la interfaz, las estructuras y los límites del módulo que gestiona el uso de un recurso compartido por varias pantallas. Es el primer lugar para comprobar qué entradas y salidas utiliza quien lo llama.

nucleo_c/obstaculos.c. Implementa el módulo que analiza color de piso y clasifica zonas de la imagen. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/corredor.c. Implementa el módulo que proyecta la huella del vehículo sobre la imagen para evaluar la zona de paso. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/cualquiera.c. Implementa el módulo que procesa el modo de detección de cualquier objeto del modelo. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/fusion.c. Implementa el módulo que combina las señales de color y del modelo de objetos para una decisión. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/mascara_vision.c. Implementa el módulo que gestiona una máscara para excluir artefactos fijos de la imagen. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/movimiento_piso.c. Implementa el módulo que compara el movimiento de bloques del piso entre cuadros. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/movimiento_visual.c. Implementa el módulo que estima giro visual mediante cuadros consecutivos. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/paquete_celdas.c. Implementa el módulo que construye e interpreta el paquete UDP de promedios de color por celda. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/pisos_sinteticos.c. Implementa el módulo que genera escenas sintéticas de piso para pruebas en computadora. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/prueba_obstaculo.c. Implementa el módulo que calcula métricas de retraso y distancia en pruebas de parada por cámara. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/latencia_video.c. Implementa el módulo que compara tiempos de respuesta del control con y sin video. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/video_stats.c. Implementa el módulo que reúne estadísticas temporales de llegada de cuadros. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

nucleo_c/sesion_compartida.c. Implementa el módulo que gestiona el uso de un recurso compartido por varias pantallas. Lea sus funciones junto con la cabecera homónima y sus pruebas para entender las condiciones, los cálculos y los casos límite.

17.6 Compilación y conexión entre C y Android

nucleo_c/Makefile. Define objetivos de compilación y pruebas del núcleo en computadora. Es útil para conocer qué archivos se enlazan en cada ejecutable y cómo se ejecuta la suite.

app_android/app/src/main/cpp/CMakeLists.txt. Declara la construcción de la biblioteca nativa de Android y sus archivos de C. Si se agrega un módulo nativo, debe comprobarse su inclusión aquí.

app_android/app/src/main/cpp/puente.c. Implementa el puente JNI entre Kotlin y el núcleo en C. Traduce llamadas, tipos y resultados entre ambos entornos; una discrepancia de firmas puede impedir que la aplicación use el módulo nativo.

app_android/app/build.gradle.kts. Configura el módulo Android, sus versiones de SDK, dependencias y compilación nativa. Debe revisarse junto con CMake y la configuración general de Gradle.

app_android/app/src/main/AndroidManifest.xml. Declara componentes y configuración de la aplicación para Android. Es la referencia para saber qué actividades y capacidades se registran en esta revisión.

17.7 Android: archivos Kotlin de aplicación

AcercaFragment.kt. Presenta información del proyecto, versión y licencias. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

Ajustes.kt. Almacena preferencias generales de la aplicación. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

AjustesFragment.kt. Ofrece la pantalla para consultar o modificar ajustes generales. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

AlmacenRutas.kt. Guarda rutas con nombre, borrador e historial en el teléfono. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

AppCarro.kt. Inicializa recursos de aplicación compartidos entre pantallas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

AsistenteFragment.kt. Guía la primera conexión y presenta los modos de uso. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

AyudaFragment.kt. Muestra ayuda y permite volver al asistente. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

BenchmarkActivity.kt. Presenta la medición de rendimiento del detector de objetos. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

BenchmarkObstaculos.kt. Organiza la medición de rendimiento del detector por color. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

CalibracionActivity.kt. Guía la medición de parámetros para convertir movimiento en tiempo. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

CalibracionGuardada.kt. Persiste y recupera la calibración del vehículo. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

CalibrarGiroActivity.kt. Dirige el procedimiento de calibración de giro y presenta su diagnóstico. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

CamaraObstaculos.kt. Procesa imágenes para detección de obstáculos y permite utilizar la cámara del teléfono como fuente. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ConfigVision.kt. Administra los parámetros ajustables del procesamiento visual. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ControlLogica.kt. Determina qué comando de control enviar y cuándo. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

CuadriculaView.kt. Dibuja una cuadrícula de interacción para rutas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

DetectorTflite.kt. Ejecuta el detector de objetos mediante TensorFlow Lite. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

Diseno.kt. Reúne componentes y decisiones visuales reutilizables. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

EditorRutaView.kt. Muestra la cuadrícula de edición y el recorrido simulado. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

EjecutarRutaFragment.kt. Presenta la ejecución de ruta, el estado y el video. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

Enlace.kt. Implementa el enlace WebSocket de control. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

EstadisticaModo.kt. Registra estadísticas para comparar modos de video. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

Estadisticas.kt. Resume listas de tiempos de medición. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

IlustracionCalibracion.kt. Dibuja las ilustraciones usadas durante la calibración. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ImagenesPrueba.kt. Proporciona imágenes JPEG para pruebas de rendimiento. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

InfoDispositivo.kt. Recoge datos del teléfono usados en reportes de rendimiento. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

InicioFragment.kt. Presenta estado de carro y cámara, conexión y accesos. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

MainActivity.kt. Actúa como armazón principal y hospeda la navegación entre pantallas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ManualFragment.kt. Presenta video y controles de conducción manual. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

MasFragment.kt. Agrupa accesos a calibración, diagnóstico, ajustes y ayuda. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

Mensajes.kt. Centraliza avisos y mensajes de error. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ModeloEnHilo.kt. Ejecuta el modelo de objetos en un hilo separado del detector de color. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

ModoSemiautonomo.kt. Coordina el ciclo del navegador nativo con los comandos que se envían al carro. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

PantallasLog.kt. Registra eventos asociados a la apertura de pantallas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

PruebaFinalFragment.kt. Presenta una secuencia guiada de pruebas del carro. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

RutasFragment.kt. Permite preparar, simular y ejecutar rutas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

RutasPredefinidas.kt. Define trazados preestablecidos usados en pruebas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

SensoresActivity.kt. Presenta el estado y diagnóstico del giroscopio y el sensor de distancia. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

SesionCarro.kt. Comparte la conexión de control entre pantallas. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

SesionVideo.kt. Comparte una sesión de video entre consumidores. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

VideoCarro.kt. Recibe el video de la cámara del carro por la red. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

VisionActivity.kt. Muestra diagnóstico visual y controles de ajuste del detector. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

VisorJpeg.kt. Presenta cuadros JPEG de la sesión de video. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

VistaDeteccion.kt. Dibuja la imagen y las marcas de detección. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

VistaNumerosDeteccion.kt. Superpone indicadores numéricos de detección sobre el video. Para comprender su comportamiento completo, revise sus llamadas a las sesiones, vistas y módulos que utiliza; la pantalla por sí sola no verifica la respuesta del carro.

17.8 Android: declaraciones para el núcleo nativo

CalGiroNativa.kt. Declara las llamadas de Kotlin al módulo calgiro del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

CalibracionNativa.kt. Declara las llamadas de Kotlin al módulo calibracion del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

CorredorNativa.kt. Declara las llamadas de Kotlin al módulo corredor del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

CualquieraNativa.kt. Declara las llamadas de Kotlin al módulo cualquiera del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

FlujoNativa.kt. Declara las llamadas de Kotlin al módulo movimiento_piso del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

FusionNativa.kt. Declara las llamadas de Kotlin al módulo fusion del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

LatenciaVideoNativa.kt. Declara las llamadas de Kotlin al módulo latencia_video del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

MovimientoVisualNativa.kt. Declara las llamadas de Kotlin al módulo movimiento_visual del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

NavegadorNativa.kt. Declara las llamadas de Kotlin al módulo navegador del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

ObstaculosNativa.kt. Declara las llamadas de Kotlin al módulo obstaculos del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

PaqueteCeldasNativa.kt. Declara las llamadas de Kotlin al módulo paquete_celdas del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

PruebaObstaculoNativa.kt. Declara las llamadas de Kotlin al módulo prueba_obstaculo del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

RutaEditorNativa.kt. Declara las llamadas de Kotlin al módulo ruta_editor del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

SesionNativa.kt. Declara las llamadas de Kotlin al módulo sesion_compartida del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

SimRutaNativa.kt. Declara las llamadas de Kotlin al módulo sim_ruta del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

TrayectoriaNativa.kt. Declara las llamadas de Kotlin al módulo trayectoria del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

VideoStatsNativa.kt. Declara las llamadas de Kotlin al módulo video_stats del núcleo C. La declaración debe corresponder con el puente JNI y la interfaz C; consulte los tres puntos al cambiar parámetros o tipos.

17.9 Pruebas, recursos y archivos auxiliares

nucleo_c/pruebas_*.c. Cada archivo agrupa pruebas del módulo o escenario indicado en su nombre. Por ejemplo, pruebas_navegador.c examina el navegador y pruebas_reincorporacion_j15.c el escenario de reincorporación. Son pruebas de software: no constituyen nuevas corridas físicas.

app_android/app/src/androidTest/. Contiene pruebas instrumentadas de Android. Su ejecución y resultados dependen del dispositivo o emulador y de la configuración empleada.

app_android/app/src/main/res/layout/ y layout-land/. Definen la composición visual de pantallas y sus variantes de orientación. Se relacionan con las clases de actividad, fragmento y vista que cargan esos recursos.

app_android/app/src/main/res/values/ y values-night/. Reúnen textos, colores, dimensiones y estilos, incluidas variantes de tema. Separar estos recursos de la lógica facilita mantener la presentación.

app_android/app/src/main/res/drawable/ y font/. Contienen iconos, formas y fuentes de la interfaz. Son recursos de presentación; no implementan las decisiones de navegación ni control.

app_android/app/src/main/assets/efficientdet_lite0.tflite. Es el archivo del modelo de objetos usado por el detector. La presencia del modelo no demuestra su exactitud para cualquier objeto o entorno.

scripts/. Contiene herramientas de compilación, carga, verificación y pruebas. Antes de ejecutar cada herramienta deben leerse sus argumentos y efectos: algunas interactúan con dispositivos o pueden escribir firmware.

README.md y archivos README de subproyectos. Describen puntos de entrada y procedimientos para el conjunto o una carpeta concreta. Cuando exista una discrepancia, compare su fecha y alcance con el código y los reportes de la revisión.

carpeta del manual. Contiene capítulos, generador, auditoría de afirmaciones y el PDF de este manual. La documentación explica el sistema, pero no reemplaza el código ni demuestra que una versión esté instalada en el equipo físico.

17.10 Herramientas de apoyo, archivo por archivo

scripts/analizar_obj_any.py. Analiza registros de detección de cualquier objeto y calcula indicadores por umbral y tipo de pasada.

scripts/buscar_nombre_anterior.py. Busca referencias al nombre anterior del proyecto en los archivos incluidos en su búsqueda.

scripts/camara_falsa.py. Simula servicios de cámara con imágenes sintéticas para probar la aplicación sin el vehículo.

scripts/camara_falsa_udp.c. Simula la comunicación UDP del modo de celdas de la cámara.

scripts/cargar.ps1. Compila y, salvo que se limite con -SoloCompilar, puede cargar firmware; algunas identificaciones automáticas pueden sobrescribir la placa con una sonda.

scripts/compilar_app.ps1. Compila la aplicación de depuración y presenta un resumen de avisos y errores.

scripts/contraste.py. Comprueba el contraste de combinaciones seleccionadas de colores de la aplicación.

scripts/generar_fuentes_inter.py. Prepara versiones estáticas de la fuente Inter para los recursos de la interfaz.

scripts/instalar_app_adb.py. Instala la aplicación por ADB e interactúa con avisos del teléfono; modifica el dispositivo conectado.

scripts/probar_modos_video.py. Prueba los modos de video de la aplicación con una cámara falsa y un teléfono mediante ADB.

scripts/probar_pantallas.py. Abre pantallas mediante ADB y revisa registros para detectar errores de apertura.

scripts/probar_version.py. Comprueba los lectores de versión mediante pruebas sin USB ni teléfono.

scripts/qa_captura.py. Abre una pantalla en el teléfono mediante ADB y guarda una captura para inspección visual.

scripts/verificar_pagina.py. Compara la página integrada del firmware con las huellas de referencia.

scripts/verificar_sensores.py. Realiza consultas físicas de sensores por puerto serie; requiere el dispositivo y no se ejecutó para esta edición.

scripts/verificar_version.py. Compara las versiones disponibles del firmware y de la aplicación con Git; su opción de solo APK evita la lectura del puerto serie.

scripts/sonda_placa/sonda_placa.ino. Programa mínimo utilizado para identificar una placa en determinados procesos de carga. Al grabarse sustituye el programa previamente instalado.

17.11 Pruebas del núcleo, archivo por archivo

pruebas_calgiro.c. Reúne casos relacionados con calibración de giro. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_calibracion.c. Reúne casos relacionados con conversión de medidas y tiempos de calibración. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_celdas.c. Reúne casos relacionados con codificación y lectura del paquete de celdas. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_corredor.c. Reúne casos relacionados con geometría del corredor. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_cualquiera.c. Reúne casos relacionados con modo de cualquier objeto. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_fusion.c. Reúne casos relacionados con reglas de fusión de señales. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_latencia_video.c. Reúne casos relacionados con cálculo de latencia con y sin video. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_mascara.c. Reúne casos relacionados con máscara de visión. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_movimiento.c. Reúne casos relacionados con estimación de movimiento visual. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_movimiento_piso.c. Reúne casos relacionados con flujo del piso. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_navegador.c. Reúne casos relacionados con estados y decisiones del navegador. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_obstaculos.c. Reúne casos relacionados con detección por color del piso. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_piso.c. Reúne casos relacionados con comportamiento del detector ante escenas de piso. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_pisos_dificiles.c. Reúne casos relacionados con escenarios sintéticos de piso difíciles. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_prueba_obstaculo.c. Reúne casos relacionados con cálculos de la prueba de obstáculos. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_reincorporacion_j15.c. Reúne casos relacionados con escenarios de reincorporación J15. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_ruta_editor.c. Reúne casos relacionados con operaciones del editor de ruta. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_sesion_compartida.c. Reúne casos relacionados con gestión de sesiones compartidas. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_sim_ruta.c. Reúne casos relacionados con ejecución de rutas simuladas. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_trayectoria.c. Reúne casos relacionados con conversión de ruta en pasos. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

pruebas_video_stats.c. Reúne casos relacionados con estadísticas de cuadros de video. Sus resultados corresponden a los datos y condiciones de la prueba de software, no a una medición física nueva.

17.12 Lectura guiada: del comando a los motores

En `carro.h` se distinguen dos grupos de funciones. Las operaciones con prefijo `hw_` describen servicios que la plataforma debe proporcionar, como aplicar PWM y leer el tiempo. Las operaciones con prefijo `carro_` describen la lógica del controlador. Esta frontera permite sustituir el hardware por implementaciones de prueba sin cambiar las decisiones centrales.

El recorrido comienza con `carro_encolar_comando`, continúa con `carro_procesar_cola` y llega a la interpretación de la orden. `carro_detener` representa una acción de parada y `carro_vigilar_comunicacion` comprueba el plazo de vigilancia. Para entender una incidencia hay que revisar también cómo el archivo Arduino invoca esas funciones y qué llamadas de hardware realiza: leer una sola función fuera de contexto puede ocultar condiciones de ejecución.

Al modificar el procesamiento de mensajes, compruebe prioridad de STOP, saturación de cola, cambios de perfil y semántica del watchdog. El tratamiento de PING y de otros

textos no es idéntico. Una prueba en computadora ayuda a detectar regresiones de lógica; una prueba física posterior verifica la integración con el hardware.

17.13 Lectura guiada: sensores y giro

`sensores.h` declara `sensores_iniciar`, `sensores_actualizar` y funciones para recibir consultas o informar métricas del bucle. `sensores.c` coordina módulos especializados: `imu.c` calcula orientación a partir de velocidad angular; `tof.c` gestiona las lecturas de distancia; `giro.c` controla una maniobra angular. Cada módulo tiene su propia cabecera con la interfaz y los estados necesarios.

La diferencia entre sensor disponible y lectura válida es fundamental. Un dispositivo detectado en I2C puede no producir una distancia válida en una situación concreta. Por eso la telemetría incluye información de estado además de valores. Antes de cambiar un umbral de parada, hay que revisar la validez, el perfil y la autoridad de parada del láser, además del valor numérico.

17.14 Lectura guiada: de la ruta a la decisión

`trayectoria.h` declara variantes de `trayectoria_calcular` que convierten puntos en pasos y aceptan distintas condiciones de orientación o calibración. La salida no es una posición real medida: es una secuencia calculada a partir de la ruta y los parámetros. Si una trayectoria es incorrecta, revise primero los puntos, la orientación de partida y la calibración utilizada.

`navegador.h` declara el estado `Navegador` y operaciones como `navegador_iniciar`, `navegador_poner_calibracion`, `navegador_cargar_ruta`, `navegador_comenzar` y `navegador_cancelar`. Las funciones de aporte reciben información de sensores y giros. `navegador.c` usa esos datos y el tiempo para avanzar por los estados. En Android, `ModoSemiautonomo.kt` coordina la llamada al núcleo y el envío de la orden resultante.

Para seguir un fallo, registre entrada, estado anterior, tiempo, salida y estado siguiente. Compare después con `pruebas_navegador.c` o con una simulación adecuada. El simulador utiliza el navegador real con un entorno modelado; sus resultados no prueban por sí mismos el comportamiento del carro físico.

17.15 Lectura guiada: imagen, color y fusión

`obstaculos.h` expone configuración, estructuras y operaciones para analizar imágenes o celdas. `obstaculos.c` implementa el modelo y el análisis. El aprendizaje utiliza un acumulador y produce un modelo de piso; por ello la escena que se presenta durante el aprendizaje importa. Si una superficie incorrecta se incorpora al modelo, una clasificación posterior puede resultar engañosa.

`fusion.h` declara la configuración de clases, la conversión de cajas a zonas y funciones de decisión como `fusion_decidir`. `fusion.c` combina el resultado del detector con las detecciones disponibles. Conviene registrar qué entradas estaban vigentes y qué opciones estaban activas; ver una caja dibujada no demuestra que se usara exactamente la misma información en una decisión anterior.

`DetectorTflite.kt` ejecuta el modelo y `ModeloEnHilo.kt` lo organiza en un hilo propio. El procesamiento de imagen y las reglas de fusión tienen ritmos y datos distintos. Al

analizar latencia, identifique el instante de captura, procesamiento y decisión, y no atribuya a toda la cadena una medición de una sola etapa.

17.16 Lectura guiada: Kotlin, JNI y C

Los archivos terminados en `Nativa.kt` declaran operaciones disponibles desde Kotlin. `puente.c` enlaza esas llamadas con las funciones del núcleo y `CMakeLists.txt` incluye los archivos C en la biblioteca nativa. Para añadir un parámetro a una operación hay que revisar coherentemente los tipos y las firmas de los tres lugares, además de quienes llamen a la función.

Una compilación correcta ayuda a detectar algunas incompatibilidades, pero no sustituye las pruebas de comportamiento. Después de un cambio en el puente, conviene comprobar también los valores límite y la gestión de errores desde la aplicación y desde las pruebas del núcleo.